Radar Human Activity Detection

Jean Jacques Chirat
Preston Bowyer
William Jachimiak
Aiden George

FINAL REPORT

REVISION – 1
26 April 2026

Radar Human Activity Detection

Jean Jacques Chirat
Preston Bowyer
William Jachimiak
Aiden George

CONCEPT OF OPERATIONS

CONCEPT OF OPERATIONS
FOR
Human Activity Radar Detector

TEAM 4

APPROVED BY:

_____
Project Leader Date

_____
Prof. Kalafatis Date

_____
T/A Date

Change Record

Rev.	Date	Originator	Approvals	Description
1	9/17/2025	Human Activity Radar Detector Team		Draft Release
2	4/26/2025	Human Activity Radar Detector Team		Final Report for ECEN 404

Table of Contents

1. Executive Summary 8
2. Introduction 8
2.1. Background 8
2.2. Overview 9
2.3. Referenced Documents and Standards 9
3. Operating Concept 10
3.1. Scope 10
3.2. Operational Description and Constraints 10
3.3. System Description 10
3.4. Modes of Operations 11
3.5. Users 11
3.6. Support 11
4. Scenario(s) 12
4.1. Assisted Living Fall Detection 12
4.2. Gym Fall Detection 12
4.3. Child/Animal in Car Detection 12
5. Analysis 12
5.1. Summary of Proposed Improvements 12
5.2. Disadvantages and Limitations 13
5.3. Alternatives 13
5.4. Impact 13

List of Tables

List of Figures

1. Executive Summary

Tracking the activities of an individual in an assisted-living home is essential in ensuring their continued physical well-being. However, doing so often requires means that unintentionally violate the privacy rights of these individuals, such as the installation of equipment that collects visual, audio, and other forms of data. These are usually installed in private areas in which the individual cannot receive assistance in a timely manner. As such, it is critical that a product be developed that both reliably detects & alerts the proper personnel for emergency assistance and does not intrude on individual privacy. This product is designed to collect data using radar signal reflections to assess the received signal. Using a machine learning model trained on collected data sets of falling individuals, the fall signal will be continuously classified over a period of ten to twenty seconds. If the pattern persists, an alert will be transmitted to a mobile application interface for assisted-living personnel so that further emergency action can be taken. The scope of this project is to provide a prototype that increases the efficiency of response times by assisted-care workers at a minimal level of personal privacy violation.

2. Introduction

This document introduces the human activity radar detector, which monitors private areas in assisted-living homes for signs of fall injury in order to alert on-site personnel for further emergency care. The module will reduce the frequency and time between injury and the arrival of assistance and will ensure that the privacy of vulnerable individuals in these spaces is not violated.

2.1. Background

Injury from falling is common among the elderly in assisted-living homes, and such injuries can cause further physical ailments if the fall goes unnoticed. According to a collection of 18 studies on elderly individuals living in nursing homes, the incidence of falls is 43% [1]. This means that a significant minority of the elderly in these facilities have experienced some form of injury from falling. Therefore, it is critical to develop a product that minimizes the potential physical impairments that result from these injuries.

The protection of individual privacy is another key aim of this project. In recent times, the usage of camera systems for home security has led to privacy failures and breaches of trust on the part of service providers. For example, in 2023, the home security company Ring did not invest in adequate privacy and security provisions of customer data, which led to the misuse of the data by employees [2]. This presents a serious risk for users of home cameras, especially for those in assisted care who are more vulnerable, and a need for an alternative that can accomplish the same tasks without violating individual privacy. This project aims to develop a system that utilizes minimally invasive data collection, which will ensure the security of users' privacy.

2.2. Overview

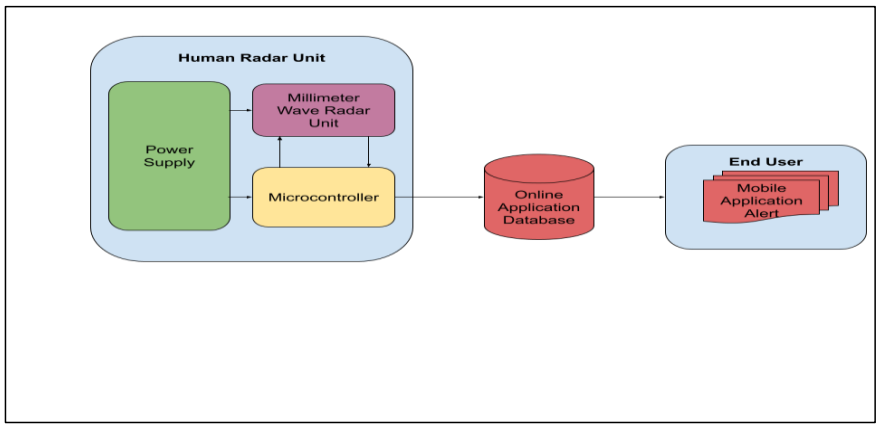


Figure 1: Human Activity Radar Detector

This system will be used to monitor a set location for people falling. This will be done by three primary components within the Human Radar Unit: the power supply, the microcontroller, and a mobile app interface to display the location of the received signal. The human activity radar detector will be outfitted with a radar module, which includes transmit and receive antennas, to detect objects at a range of eight to ten meters. This radar module will continually transmit & receive signals, which, using an onboard signal processing module, will send digital data to a microcontroller. The microcontroller will be programmed with a machine learning algorithm that will be trained on data sets of fall injury radar patterns. By classifying the received signal, the module will alert the on-site personnel through a mobile app interface so that they can provide the necessary care.

2.3. Referenced Documents and Standards

[1] Shao L, Shi Y, Xie XY, Wang Z, Wang ZA, Zhang JE. Incidence and Risk Factors of Falls Among Older People in Nursing Homes: Systematic Review and Meta-Analysis. J Am Med Dir Assoc. 2023 Nov;24(11):1708-1717. doi: 10.1016/j.jamda.2023.06.002. Epub 2023 Jul 8. PMID: 37433427.

[2] Staff, BCP, et al. "Ring's Privacy Failures Led to Spying and Harassment through Home Security Cameras." *Consumer Advice*, 30 Oct. 2023, consumer.ftc.gov/consumer-alerts/2023/05/rings-privacy-failures-led-spying-harassment-through-home-security-cameras.

3. Operating Concept

3.1. Scope

The Human Activity Radar Detector (HARD) will be used in residential and assisted living settings to detect human falls within a short distance (below 10m) using a high-frequency radar sensor. The radar sensor mounted on the wall will constantly be sending data to a control unit, where a microcontroller running a classification machine learning model will determine if there has been a fall injury. Once a fall injury has been detected, the microcontroller will communicate with an app via Wi-Fi to send out an alert to caretakers/family and emergency services.

3.2. Operational Description and Constraints

The Human Activity Radar Detector will be installed in spaces where assisted care is at the lowest level, such as bathrooms, bedrooms, etc. The module will transmit & receive signal reflections that will be processed to determine the occurrence of a falling injury, verify the certainty that the injury has in fact occurred, and alert the nearby assisted-care worker through an app interface. Below are listed the constraints of this design:

- The app interface must be able to identify the designated room location of a signal transmission in case of an emergency.
- The module must be able to operate over the Wi-Fi network or otherwise be able to access an internet connection.
- The machine learning algorithm for detecting signal patterns will be trained on a predetermined set of data, which cannot be modified in the future.
- The system must be able to operate using a 14V battery that will be regulated by on-board circuitry.
- The system must continually classify affirmative signals for a period of 10-20 seconds before alerting personnel to prevent false alarms from triggering.

3.3. System Description

The radar detection system will be defined by three central parts: the radar detector, the microcontroller programmed with a machine learning algorithm, and the mobile application user interface. Power will be provided by a peripheral power management system. The radar detector will deliver digitally processed FMCW data to the microcontroller, which will classify the signal using a machine learning algorithm. The algorithm will decide whether the signal resembles a fall injury. If so, a signal will be transmitted over Wi-Fi to the user interface to indicate that a fall has occurred.

3.4. Modes of Operations

The system will have one primary mode of operation, in which detected falls are tracked with an alert sent to a mobile device. This will consist of three primary actions performed by the system: detection, validation, and finally alerting the user.

Detection
The system will emit & process signals continuously to identify whether an injury pattern has occurred.

Validation
The system will continue to process signals as before; however, the system will also maintain a counter that will verify that an injured person has remained on the floor after the fall.

Alert
The system will send an alert to the user interface to inform assisted-care personnel in the area that an injury has occurred.

3.5. Users

This system will primarily be used by individuals who are at risk of serious injury from falls and their immediate family or caretakers. The level of training required to use this system will be very low, with the only requirements being the installation of a mobile app and the mounting of the system on a wall. This system will benefit both at-risk individuals and their families, providing both of those groups with the peace of mind that if something bad were to happen, they would be informed of it in a timely manner, along with the appropriate medical personnel.

3.6. Support

This system will include a user manual in the box detailing how to install the physical device and the phone app. If users are having issues with the app, they will be able to either call a support number with their issue or send an email relaying the issue to the manufacturer.

4. Scenario(s)

4.1. Assisted Living Fall Detection

The Human Activity Radar Detector's primary use is for fall detection in assisted living environments. Residents in assisted living are at high risk of falls, which, if left unattended, can result in compounded side effects. The proposed radar system would provide continuous, non-invasive monitoring of desired areas in residential care homes (possibly in bedrooms/bathrooms where staff are less present). When falls are detected, the system will alert the staff via a phone/web app, ensuring a rapid response.

4.2. Gym Fall Detection

Another possible application of this system is to work within public gyms. Given the irregular hours that gyms can be open, having personnel on site constantly monitoring issues could be difficult. Areas or exercises where fall detection could be applicable include the squat rack, deadlift, power clean, and other such exercises with a full range of motion. Having an automated alert system could help employees watch for possible life-threatening issues while reducing mental workload.

4.3. Child/Animal in Car Detection

Another application of radar detection could be detecting children or animals inside a locked car. There have been numerous cases globally of child or animal deaths due to being stuck in locked cars on a hot day. This issue is prevalent enough that, in the coming years, the EU will require cars to have radar detection systems inside for this exact purpose. There is no practical way for any police body to monitor all cars to protect against this issue. Having a system to check for this can ensure more child and animal lives are saved each year.

5. Analysis

5.1. Summary of Proposed Improvements

- The system will operate off battery power for its main configuration; this will allow placement in locations where a power source is not available.
- Ease of installation: Given that the system's logic is based on a machine learning model, user input will be minimal during both installation and operation.
- Cybersecurity integration: In order to prevent wireless communication that may retrieve user data, integrating basic cybersecurity into the system would greatly advance the stated purpose of protecting users' privacy.

5.2. Disadvantages and Limitations

- Need to ensure that batteries are adequately charged.
- Limited directionality compared to a camera-operated system.
- High cost for FMCW radar sensors.

5.3. Alternatives

Alternatives to the millimeter wave radar detection solution include, but are not limited to:

- Constant monitoring by attending caretakers
- Off-the-shelf camera monitoring systems
- The same unit, but with a wired power solution
- Life-Alert style button
- Smartwatch fall detection alerts

5.4. Impact

This design is largely focused on reducing the impact of data collection on personal privacy; however, there remains a minimal level of data that must be collected from the operating environment in order for the design to function. This does not pose any significant risk to the user's personal information or privacy. In addition, the programmed algorithm does not feature the ability to receive updates, modifications, or any devices that record data that are not explicitly stated in this document, making the user's data secure from malpractice.

Human Activity Radar Detector

William Jachimiak
Jean Jacques Chirat
Preston Bowyer
Aiden George

INTERFACE CONTROL DOCUMENT

REVISION 2
26 April 2026

INTERFACE CONTROL DOCUMENT
FOR
Human Activity Radar Detector

PREPARED BY:

_____
Author Date

APPROVED BY:

_____
Project Leader Date

_____
John Lusher II, P.E. Date

_____
T/A Date

Change Record

Rev.	Date	Originator	Approvals	Description
1	10/4/2025	Human Activity Radar Detector Team		Draft Release
2	12/06/2025	Human Activity Radar Detector Team		Raspberry Pi Added
3	4/26/2026	Human Activity Radar Detector Team		Final Report for ECEN 404

Table of Contents

1. Overview 20
2. References and Definitions 20
2.1. References 20
2.2. Definitions 20
3. Physical Interface 20
3.1. Weight 20
3.2. Dimensions 21
3.2.1. Dimension of SBC subsystem 21
3.2.2. Dimension of Radar Board subsystem 21
3.2.3. Dimension of Power Supply Subsystem 21
3.3. Mounting Locations 21
4. Thermal Interface 22
5. Electrical Interface 22
5.1. Primary Input Power 22
5.2. Voltage and Current Levels 22
5.3. Signal Interfaces 23
5.4. User Control Interface 23
6. Communications / Device Interface Protocols 23
6.1. Wireless Communications (WiFi) 23
6.2. Device Peripheral Interface 23

List of Tables

List of Figures

Figure 1: Power Distribution System **3**

1. Overview

The Interface Control Document (ICD) for the Human Activity Radar Detector will provide more information on how the subsystems in the concept of operations and the functional system requirements (FSR) will be produced. It will also include more information on how each subsystem will interact with the others.

2. References and Definitions

2.1. References

"MMWAVE-SDK Software Development Kit (SDK) | TI.com." *Ti.com*, 2022, www.ti.com/tool/MMWAVE-SDK.

2.2. Definitions

PCB	Printed Circuit Board
mA	Milliamp
mW	Milliwatt
GHz	Gigahertz (1,000,000,000 Hz)
TBD	To Be Determined
UART	Universal Asynchronous Receiver-Transmitter
ML	Machine Learning
SBC	Single Board Computer

3. Physical Interface

3.1. Weight

Component	Weight (kg)
Raspberry Pi 5	0.046
External Rechargeable Battery Pack	1
Radome	0.25
Radar Chip Board	0.119

Table 1: *System Component Weights*

3.2. Dimensions

	Length (mm)	Width (mm)	Height (mm)
Raspberry Pi 5	85.6	56.5	21

Table 2*: SBC Dimensions*

3.2.1. Dimension of SBC subsystem

	Length (mm)	Width (mm)	Height (mm)
TI IWR1642	76.2	61.0	15.2

Table 3*: Radar Board Dimensions*

3.2.2. Dimension of Radar Board subsystem

	Length (in)	Width (in)	Height (in)
Power Supply PCB	5.5	6	2

Table 4*: Power Supply Dimensions*

3.2.3. Dimension of Power Supply Subsystem

3.3. Mounting Locations

The device may be mounted in any location on a wall that is not exposed to electrical hazards, such as water. For placement in areas of operation with unavoidable contact with water, such as a bathroom, the device should be at a high altitude to prevent component damage.

4. Thermal Interface

Our device is expected to use a small heat sink for the purpose of cooling the single-board computer, which is the main heat-generating component of the system. All other components that generate a non-trivial amount of heat, such as the radar board, are expected to be adequately cooled by natural air cooling.

5. Electrical Interface

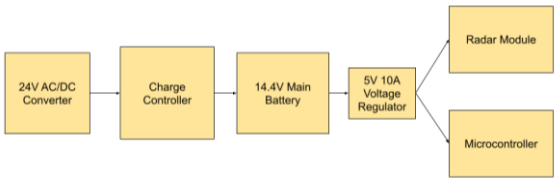


Figure 1: Power Distribution System

Commented [1]: Revise Figure

5.1. *Primary Input Power*

The primary power source for the system will be an internal rechargeable lithium-ion battery pack with a nominal voltage of 14.4V and a 3000mAh capacity. This battery will be charged using a DC 24V 4A input, which will be stepped down using constant current charging from the charger IC. As both the radar module and microcontroller have high power requirements, the device is expected to be able to run off battery power for at least 4 hours before needing to charge.

5.2. *Voltage and Current Levels*

Component	Voltage	Current (Max)	Power (Max)
Charge Controller (Charging Output)	14V	4A	56W
Radar Unit	5V	5A	25W
Raspberry Pi 5	5.1V	5A	25.5W

Table 5: Device Power Consumption

5.3. Signal Interfaces

Our system does not provide the user with a way to interact with the signals collected by the module to ensure the simplicity of design.

5.4. User Control Interface

The user will be able to interact with the system through the mobile/web application interface, where the individual modules will be instantiated, and the modules can inform the user in the case of a detection event.

6. Communications / Device Interface Protocols

6.1. Wireless Communications (WiFi)

The microprocessor includes a built-in Wi-Fi system with integrated 2.4 GHz, 802.11 b/g/n Wi-Fi. Through this, the output determined by the ML model will be published to a cloud service that the application is subscribed to.

6.2. Device Peripheral Interface

The TI IWR1642 radar chip communicates with and sends data to the Raspberry Pi 5 through a USB port.

Human Activity Radar Detector

William Jachimiak
Jean Jacques Chirat
Preston Bowyer
Aiden George

FUNCTIONAL SYSTEM REQUIREMENTS

REVISION 3
26 April 2026

FUNCTIONAL SYSTEM REQUIREMENTS
FOR
Human Activity Radar Detector

PREPARED BY:

_____
Author Date

APPROVED BY:

_____
Project Leader Date

_____
John Lusher, P.E. Date

_____
T/A Date

Change Record

Rev.	Date	Originator	Approvals	Description
1	10/4/2025	Human Activity Radar Detector Team		Draft Release
2	12/06/2025	Human Activity Radar Detector Team		Raspberry Pi Added
3	4/26/2026	Human Activity Radar Detector Team		Final Report for ECEN 404

Table of Contents

1. 311.1.
311.2.
31**2.**
312.1.
312.2. **Error! Bookmark not defined.**

List of Tables

List of Figures

1. Introduction

1.1. Purpose and Scope

This document will prescribe the system requirements of this design in real-world operation. The purpose of the Human Activity Radar Detector (HARD) is to improve the efficiency of response times & privacy protection in assisted-care homes in the event of a fall injury. The system will improve response times by actively identifying a fall injury event using machine learning. By using a trained machine learning model, the system will classify fall injuries at a highly reliable rate. The system will provide the user with information about the location of detection in the event of injury. The system will collect the minimal amount of data necessary to ensure efficient detection. The system should ensure up to ten hours of typical operation before a battery recharge is necessary.

1.2. Responsibility and Change Authority

The functional requirements of the system should only be modified with the consensus of the team members. The details of implementation & methodology of subsystems may be modified upon the agreement of the team members. The team leader, William Jachimiak, will be responsible for making sure that the functional requirements of the system are met. The people responsible for each subsystem are listed below:

- William Jachimiak, SBC and Radar Board
- Jean Jacques Chirat, Machine learning algorithm & training
- Aiden George, Power system & distribution
- Preston Bowyer, User-end application interface

2. Applicable and Reference Documents

2.1. Applicable Documents

The following documents, of the exact issue and revision shown, form a part of this specification to the extent specified herein:

Document Number	Revision/Release Date	Document Title
47-CFR-95-M	9/20/2017	Permitting Radar Services in the 76-81 GHz Band
45-CFR-160,164	12/28/2000	Standards for Privacy of Individually Identifiable Health Information
ISO/IEC 29100:2024	02/2024	Security Techniques - Privacy Framework

2.2. Order of Precedence

In the event of a conflict between the text of this specification and an applicable document cited herein, the text of this specification takes precedence without any exceptions.

All specifications, standards, exhibits, drawings or other documents that are invoked as “applicable” in this specification are incorporated as cited. All documents that are referred to within an applicable report are considered to be for guidance and information only, except ICDs that have their relevant documents considered to be incorporated as cited.

3. Requirements

This section defines the minimum requirements that the development item(s) must meet. The requirements and constraints that apply to performance, design, interoperability, reliability, etc., of the system, are covered.

3.1. System Definition

The Human Activity Radar Detector is a fall detection system that will be able to detect a fall injury and send a notification via a web app to caretakers and family. It will be mounted to the wall in residential areas such as bedrooms and bathrooms. The four subsystems for the Human Activity Radar Detector are the power supply, SBC and Radar Board, the machine learning algorithm, and the web application.

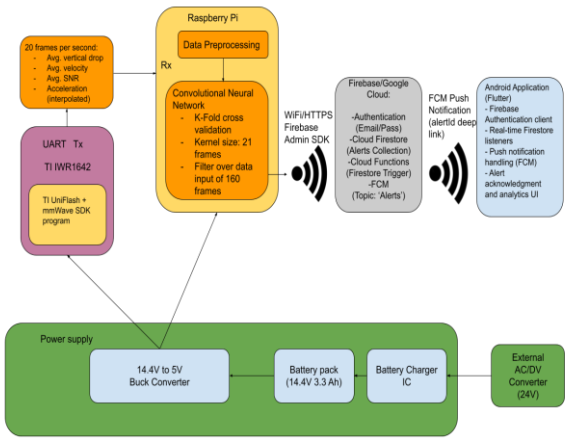


Figure 2: Block Diagram of System

The mmWave radar will transmit a signal consisting of the height component of a moving person and a range-doppler velocity component. The board will transmit this data to the Raspberry Pi 5 microprocessor, where the data will be processed through a simple neural network, with two criteria: a decrease in vertical coordinates and a downward velocity above an observed threshold. If it is decided that a fall injury occurred, the single board computer will communicate with an online database, where the status of injury will be updated and the location of the particular module will be displayed for personnel to assist the injured.

3.2. Characteristics

3.2.1. Functional / Performance Requirements

3.2.1.1. Responsiveness

The system will be able to perform detection and alert the user within ten seconds of the occurrence of the fall injury event. The algorithm execution time, in addition to network latency, should remain under the limit of 20s in all cases.

3.2.1.2. False Positive Rate

The system will have a low false positive rate of at most five to ten percent.

Rationale: A low false positive rate such as ~8% will prevent frequent disruption of other assisted-care responsibilities that on-site personnel may have.

3.2.1.3. False negative rate

The system will have a very low to negligent false negative rate, and must be below 5% by requirement..

> *Rationale: The costs & liabilities of a false negative are far more consequential than a false positive. Therefore, it is imperative that false negatives are made as infrequent as possible for a reliable usage of the system.*

3.2.1.4. Battery Life Duration

The Human Activity Radar Detector should be able to run on battery power and provide up to four hours of active operation.

> *Rationale: The Human Activity Radar Detector will be entirely powered by battery and as a system that is designed to be able to detect at any time, it should be able to run for an extended period of time that is at least 4hours.*

3.2.2. Physical Characteristics

3.2.2.1. Installation

The installation of the radar detection module shall be specified in the user manual provided. The module will be pre-programmed with the trained algorithm.

3.2.2.2. Mass

The mass of the human activity radar detector shall be less than or equal to 4 kilograms.

> *Rationale: The system should be able to be hung on drywall using drywall screws with anchors that at minimum should be able to hold roughly 6 kg. Our limit for mass is less than this amount to ensure that there is minimal risk of damaging the equipment.*

3.2.2.3. Volume Envelope

The volume envelope of the Human Activity Radar Detector shall be less than or equal to 4 inches in height, 7 inches in width, and 9 inches in length.

3.2.2.4. Mounting

The mounting information for the Human Activity Radar Detector shall be captured in the Human Activity Radar Detector ICD.

3.2.3. Electrical Characteristics

3.2.3.1. Power Consumption

The maximum peak power of the MCU and radar board shall not exceed 50W, or 10A of combined current at regulated voltage.

> *Rationale: This is to ensure that the module is not damaged by either overcurrent or overvoltage.*

3.2.3.2. Voltage Output Level

The output voltage of the system will be 5V with a 10% max variation in output level.

> *Rationale: Our radar board and microprocessor are designed to operate using 5V. This voltage level will be obtained using a stepdown buck converter and LDO before being supplied to the component.*

3.2.3.3. Battery Charging Circuit

The Human Activity Radar Detector should be able to charge the systems batteries at 4A constant current constant voltage and then shutoff at 15.4V

Rationale: Lithium-Ion batteries should charge in a constant current / constant voltage configuration to ensure the fastest charging while ensuring a higher charging current than use power.

3.2.3.4. Outputs

3.2.3.4.1 Data Output

The Human Activity Radar Detector will provide a mobile application for users to view detection alerts.

Rationale: The detection information passes directly to the caretaker's system.

3.2.3.4.2 Diagnostic Output

The Human Activity Radar Detector shall include identification for each radar module as well as the location of the fall injury.

Rationale: Eliminates the necessity of searching all module locations and pinpoints the exact location to facilitate a speedy response to the injured person.

3.2.3.4.3 Connectors

The Human Activity Radar Detector shall follow the American National Standard for Electrical Connectors ANSI C119.6-2011.

Rationale: Conform to connector standard.

3.2.3.4.4 Wiring

The Human Activity Radar Detector shall follow the guidelines set forth in the National Electric Code regarding electrical wiring.

Rationale: Conform to wiring standard.

3.2.3.4.5 Historical Output

The Human Activity Radar Detector shall store and display a timestamped history of fall detection events and diagnostic alerts.

Rationale: Enables post-event review, system performance analysis, and medical documentation if required.

3.2.3.4.6 Environmental Requirements

The Human Activity Radar Detector will operate within the minimal requirements for the environment specified in the section below.

Rationale: The HARD system will operate indoors where exposure to the elements will be minimal.

3.2.3.5. Thermal

The Human Activity Radar Detector will be able to operate in temperatures ranging from -20°C to 60°C. The module is expected to be stored in a room temperature environment, away from long-term exposure to outside conditions.

Rationale: The HARD system PCB may be damaged, or its operation hindered by exposure to abnormal temperature.

3.2.3.6. Humidity

The Human Activity Radar Detector will be able to operate in humidity ranging from 0% to 100%. The module is expected to be stored in a dry environment.

Rationale: The HARD system PCB may oxidize in a humid environment, which may seriously damage performance expectations.

3.2.4. Failure Propagation

The Human Activity Radar Detector shall not allow propagation of faults beyond the Human Activity Radar Detector user interface.

3.2.4.1. Failure Detection, Isolation, and Recovery (FDIR)

3.2.4.1.1 Built In Test (BIT)

The Human Activity Radar Detector shall be able to evaluate detection patterns and determine whether there is a failure.

3.2.4.1.1.1 BIT False Alarms

The Human Activity Radar Detector shall have a false alarm rate of less than 5 percent.

Rationale: This is a useful requirement for the purpose of limiting time spent by personnel reacting to false alarms.

3.2.4.2 System Load Tolerance

The Human Activity Radar Detector shall maintain core detection and alert functionality when subjected to high volumes of input events.

Rationale: The system may experience periods of elevated event activity due to sensor noise, environmental disturbances, or testing conditions. Maintaining functionality under high input load ensures system reliability and prevents loss of critical detection events.

3.2.5 Security

3.2.5.1 User Authentication

The Human Activity Radar Detector system shall require user authentication to access detection alerts, diagnostic information, and system status through the user interface.

Rationale: Detection alerts and diagnostic data constitute sensitive information and must be protected from unauthorized access to ensure user privacy, system integrity, and appropriate use of the system.

4. Support Requirements

The Human Activity Radar Detector requires a Wi-Fi connection and a functioning cloud database to operate, as well as a rechargeable battery with sufficient voltage level. The SBC will operate on the Wi-Fi network to transmit detection status to the cloud database.

5. Appendix A: Acronyms and Abbreviations

BIT	Built-In Test
FOV	Field of View
FSR	Functional System Requirements
GUI	Graphical User Interface
SBC	Single Board Computer
Hz	Hertz
ICD	Interface Control Document
kHz	Kilohertz (1,000 Hz)
mA	Milliamp
Ah	Ampere-hour
GHz	Gigahertz (1,000,000,000 Hz)
W	Watt
PCB	Printed Circuit Board
TBD	To Be Determined
UART	Universal Asynchronous Receiver-Transmitter

Human Activity Radar Detector

William Jachimiak

Jean Jacques Chirat

Preston Bowyer

Aiden George

EXECUTION PLAN

REVISION --

2 December 2025

Execution Plan for HARD Human Activity Radar Detector

	9-Sep	16-Sep	23-Sep	30-Sep	7-Oct	14-Oct	21-Oct	28-Oct	4-Nov	11-Nov	18-Nov	25-Nov	2-Dec
Learn and Understand the Problem													
Research and Planning										Not Started			
Write ConOps										In Progress			
Choose & order radar board										Completed			
Midterm Presentation										Late			
Write FSR & ICD													
Order Parts													
App UI Started													
Receive Parts													
Fire base cloud and database up													
Develop data preprocessing code													
Have Training Data Complete													

Complete PCB Design													
Status Update Presentation													
Automatic FCW Updates Working													
Develop code for ML													
Interface MCU to Radar													
Interface MCU to WIFI													
Evaluate ML model using data													
Apply Power Supply to PCB													
Apply MCU on PCB													
Final Presentation													
Validate Radar-MCU-WIFI													
Final Presentation													
Demo													

Human Activity Radar Detector

William Jachimiak

Jean Jacques Chirat

Preston Bowyer

Aiden George

VALIDATION PLAN

REVISION --

2 December 2025

Validation Plan for HARD Human Activity Radar Detector

FSR Section	Requirement	Description	Margin	Status
3.2.1.1	Responsiveness of system	The system will be able to perform detection and alert the user within a reliable timeframe of the occurrence of the fall injury event. The algorithm execution time, in addition to network latency, must remain under limit in all cases to ensure the safety of the user. This metric is defined as the time necessary to collect raw motion data, preprocess the data into the form suitable for machine learning, conduct inferencing based on said data, publish data to the cloud database, and notify the user via mobile app.	<20 s	Satisfactory
3.2.1.2	False positive rate	The system must have a low false positive rate in order to minimize the additional consumption of resources caused by false alarms, and to ensure that true positive predictions are proportionally reliable.	<5%	Did not meet requirement (11%)
3.2.1.3	False negative rate	The system must have a very low false negative rate in order to minimize liability for the user and ensure the reliability of the system for persistent, daily usage	<5%	Satisfactory
3.2.1.4	Battery life duration	The power supply subsystem must be able to provide the Raspberry Pi and radar board with battery life for reliable daily operation at full recharge.	>4 hrs	Satisfactory
3.2.1.5*	Maximum range	The system must be able to detect falls at and below the distance specified to ensure proper coverage in the deployed space.	7 m	Satisfactory
3.2.2.2	Mass	The mass of the full system including the Raspberry Pi, radar board, radar housing unit, custom PCB (including mass of batteries) must be within safe limits for a wall mounted installation.	<4 kg	Satisfactory
3.2.2.3	Volume envelope	The dimensions of the full system must be within a reasonable limit such that it does not obstruct the living spaces of the user or otherwise prove difficult to handle. The acceptable limit for this system is less than 4 inches in height, 7 inches in width, and 9 inches in length.	<252 cu in	Satisfactory
3.2.2.4	Mounting & installation	The device must be designed for easy installation in any non-hazardous location for electrical equipment. The product mount must support installation on any part of a wall.	X	Satisfactory
3.2.2.5	Input safety	Adherence to the recommended inputs for power & data transfer should not result in hardware damage to any component in the system.	X	Satisfactory

3.2.2.5.1	Power consumption	The maximum power consumption of the system must be at or below a reasonable limit to ensure energy efficiency for the task and the safe operation of individual components	<50 W	Satisfactory
3.2.2.5.2	Input voltage level	The input voltage of the system will be a built-in 14V as a result of the battery arrangement, which will be reduced using power electronics to a safe 5V rated for the equipment in the system. The power supply subsystem is required to remain within the safe range of input voltage dictated by parts manufacturers.	>4.9V, <5.25V	Satisfactory
3.2.2.5.3	Battery Charging Current	The battery charging IC must be able to charge the battery at 4A and safely stop battery charging at 102% of the battery pack ideal voltage. The charging current must remain within the safe range of input current dictated by battery manufactures	>3.8A, <4.2A	Satisfactory
3.2.2.6.1	Output data	The alerts must be displayed on a mobile application that provides ease of use allows for acknowledgement by users	X	Satisfactory
3.2.2.6.2	Output diagnostic	The mobile app must be able to display data reated to the fall: Confidence score, location, time, and acknowledged (true or false).	X	Satisfactory
3.2.2.6.5	Historical Output	The mobile app must have a logistics page that holds records of alerts, graphs that show trends, and other data pertinet to risk assement on rooms/residents. The length of history should not deteriorate the functionallity or responsiveness of the system	X	Satisfactory
3.2.3.1	Thermal operation	The system must operate with full capability in the specified temperature range to ensure consistent reliability in diverse environments.	-20°C to 60°C	Satisfactory
3.2.3.2	Humidity	The system must operate normally in all humidity settings to prevent damage to external parts, and is not designed to operate in areas where the components may contact water, but may operate in adjacent areas (i.e. placement in higher locations).	0% to 100%	Satisfactory
3.2.4.1.1	Built-in test (BIT)	The system must supply a history of detection accuracy, including instances of false alarms and missed alarms, for the purpose of user evaluation of system reliability.	X	Satisfactory
3.2.4.2	System Load Tolerance	The system may experience periods of elevated event activity due to various reasons. Maintaining functionality under high input load ensures system reliability and prevents loss of critical detection events.	X	Satisfactory

3.2.5.1	User Authentication	The mobile app must have strict user authentication because detection alerts and diagnostic data constitute sensitive information and must be protected from unauthorized access to ensure user privacy, system integrity, and appropriate use of the system.	X	Satisfactory
Comprehensive	Full-system demonstration	The fully integrated system must demonstrate the ability to capture motion data through the radar, infer any pattern of fall injury, and communicate with the user front-end application that a fall has occurred. This demonstration must evaluate all previous performance metrics previously mentioned.	X	Satisfactory

Human Activity Radar Detector

William Jachimiak

Jean Jacques Chirat

Preston Bowyer

Aiden George

SUBSYSTEM REPORTS

REVISION --

2 December 2025

SUBSYSTEM REPORT

FOR

Human Activity Radar Detector

PREPARED BY:

_____

Author Date

APPROVED BY:

_____

Project Leader Date

_____

John Lusher, P.E. Date

_____

T/A Date

Change Record

Rev.	Date	Originator	Approvals	Description
-	12/6/2025	Human Activity Radar Detector Team		Draft Release

Table of Contents

1 Introduction 52
2 Microprocessor Unit Subsystem Report 52
2.1 Subsystem Introduction 52
2.2 Subsystem Details 52
2.3 Subsystem Validation 54
2.4 Subsystem Conclusion 55
3 Power Supply Subsystem Report 55
3.1 Subsystem Introduction 55
3.2 Battery Charger IC 56
3.3 Voltage Regulator 57
3.4 Subsystem Initial Validation 58
3.5 Power Supply PCB 59
3.6 Power Supply Assembly and Validation 60
3.6.1 Battery Charger 61
3.6.2 Voltage Regulator 63
4 Machine Learning Subsystem Report 64
4.1 Subsystem Introduction 64
4.2 Subsystem Details 65
4.2.1 Data Formatting 65
4.2.2 Neural Network Architecture 66
4.2.3 K-Fold Cross Validation 66
4.3 Subsystem Validation 67
4.4 Subsystem Conclusion 69
5 App & Interface Subsystem Report 69
5.1 Subsystem Introduction 69
5.2 Subsystem Details 69
5.2.1 Mobile Application Infrastructure 69
5.2.2 Data Model and Storage Design 70
5.2.3 Software Architecture Stack 71
5.2.4 Application Interface and Functional Design 72

5.2.5 System Data Flow Pipeline 75
5.2.6 System Health Monitoring (Heartbeat) 76
5.3 Subsystem Validation 76
5.4 Subsystem Conclusion 79

List of Tables

Table 1: Microprocessor Subsystem Validation Plan 15
Table 2: Machine Learning Subsystem Performance Criteria 23
Table 3: App User Field Statistics 26
Table 4: Mobile Subsystem Validation Test 32

List of Figures

Figure 1: Received Configuration Confirmation from the TI Radar Board 53
Figure 2: An Output Frame showing Position, Velocity, and Number of Detected Objects 53
Figure 3: Webhook Showing the JSON file Contents 54
Figure 4: Frame Failure 55
Figure 5: Buck Converter Current Simulation 56
Figure 6: Power Supply Schematic 57
Figure 7: 5V Buck Converter Schematic 58
Figure 8: Buck Converter Voltage Simulation 58
Figure 9: Buck Converter Current Simulation 59
Figure 10: Power Supply Subsystem PCB Layout 60
Figure 11: Power Supply PCB 61
Figure 12: Battery Charging Voltage & Current Response 62
Figure 13: Battery Charging Voltage & Current 62
Figure 14: Battery Voltage 63
Figure 15: Battery Voltage vs Current 63
Figure 16: Machine Learning Subsystem Block Diagram 65
Figure 17: Training & Validation Loss Curves 66
Figure 18: 10-Fold Cross Validation Accuracy 67
Figure 19: Terminal Output for Model Training Run 68
Figure 20: HARD Mobile Subsystem Data Flow 70
Figure 21: Unacknowledged Alerts Tab 72
Figure 22: Acknowledged Alerts Tab 73
Figure 23: Logistics/Analytics Page 74
Figure 24: Login, Alert Details, and Settings Pages 75

1 Introduction

Our project will monitor the physical safety of fall injury prone individuals in assisted care using minimally invasive methods. The system collects vertical position and velocity data using a TI mmWave radar. The radar streams the data packets to a Raspberry Pi 5, which will run a machine learning detection model, which is trained on collected datasets employing the same radar module and using the same type of data. The Raspberry Pi communicates with an online Firebase database over the Wi-Fi network in order to transmit the detection result. The detection result, if positive, is displayed by an Android app which has an interface with Firebase. The app will notify the caretaker responsible with an alert displaying that a fall has occurred along with the room location in the event of an injury.

2 Microprocessor Unit Subsystem Report

2.1 Subsystem Introduction

The microprocessor subsystem of the Human Activity Radar Detector acts as a means of interfacing the TI mmwave IWR1642BOOST radar board with the machine learning subsystem and then interfacing the results of that with the app subsystem. The interface between the radar chip and the microprocessor was validated through the computer console, showing both what is being transmitted by the microprocessor and what is being received and comparing that to expected results. The interface between the app and the microprocessor was validated using a webhook displaying the JSON contents.

2.2 Subsystem Details

The microprocessor uses UART to communicate with the TI mmwave radar board using the boards built in micro usb to UART port. The TI mmwave radar board uses the Raspberry Pi sends the configuration details to the TI radar board and once it receives back positive confirmation that the board is correctly configured, it takes data from the board at 20 FPS and processes the data. This is shown in Figures 1 and 2. From there, the microprocessor sends a JSON type file over Wi-Fi whenever it receives the push of a button (which simulates a positive fall detection from the machine learning subsystem) which is shown in Figure 3.

```
b'sensorStop\nIgnored: Sensor is already stopped\n\rDone\n\rmmwDemo:/>flushCfg\nDone\n\rmmwDemo:/>dfeDataOutputMode 1\nDone\n\r
mmwDemo:/>channelCfg 15 3 0\nDone\n\rmmwDemo:/>adcCfg 2 1\nDone\n\rmmwDemo:/>adcbufCfg -1 0 1 1 0\nDone\n\rmmwDemo:/>profileCfg
 0 77 79 7 18.24 0 0 82.237 1 64 6249 0 0 30\nDone\n\rmmwDemo:/>chirpCfg 0 0 0 0 0 0 1\nDone\n\rmmwDemo:/>chirpCfg 1 1 0 0 0
0 0 2\nDone\n\rmmwDemo:/>frameCfg 0 1 128 0 50 1 0\nDone\n\rmmwDemo:/>lowPower 0 1\nDone\n\rmmwDemo:/>guiMonitor -1 1 1 0 0 0 0
\nDone\n\rmmwDemo:/>cfarCfg -1 0 2 8 4 3 0 15 1\nDone\n\rmmwDemo:/>cfarCfg -1 1 0 8 4 4 1 15 1\nDone\n\rmmwDemo:/>multiObjBeamF
orming -1 1 0.5\nDone\n\rmmwDemo:/>clutterRemoval -1 0\nDone\n\rmmwDemo:/>calibDcRangeSig -1 0 -5 8 256\nDone\n\rmmwDemo:/>exte
ndedMaxVelocity -1 0\nDone\n\rmmwDemo:/>bpmCfg -1 0 0 1\nDone\n\rmmwDemo:/>lvdsStreamCfg -1 0 0 0\nDone\n\rmmwDemo:/>compRangeB
iasAndRxChanPhase 0.0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0\nDone\n\rmmwDemo:/>measureRangeBiasAndRxChanPhase 0 1.5 0.2\nDone\n\rmmwD
emo:/>CQRxSatMonitor 0 3 4 31 0\nDone\n\rmmwDemo:/>CQSigImgMonitor 0 31 4\nDone\n\rmmwDemo:/>analogMonitor 0 0\nDone\n\rmmwDemo
:/>aoaFovCfg -1 -90 90 -90 90\nDone\n\rmmwDemo:/>cfarFovCfg -1 0 0 9.11\nDone\n\rmmwDemo:/>cfarFovCfg -1 1 -5 5.00\nDone\n\rmmw
Demo:/>calibData 0 0 0\nDone\n\rmmwDemo:/>sensorStart\nDebug: Init Calibration Status = 0x17fe\n\rDone\n\rmmwDemo:/>'
```

Figure 1: Received Configuration Confirmation from the TI Radar Board

```
headerStartIndex    = 0
totalPacketNumBytes = 320
platform            = b'000a1642'
frameNumber         = 667
timeCpuCycles       = 895127384
numDetObj           = 5
numTlv              = 3
subFrameNumber      = 0
The 1st TLV
    type 1
    len 80 bytes
The 2nd TLV
    type 7
    len 20 bytes
                  x(m)         y(m)         z(m)        v(m/s)    Com0range(m)  azimuth(deg)  elevAngle(deg)  snr(0.1dB)    noise(0.1dB)
    obj  0:    -0.122466     1.955622     0.000000     0.000000     1.959453    -3.583322           0          155          881
    obj  1:     0.612329     1.861319     0.000000     0.000000     1.959453    18.209957           0          155          881
    obj  2:     0.022267     0.176735     0.000000     0.618046     0.178132     7.180756           0          440          665
    obj  3:    -0.050100     0.170942     0.000000     0.618046     0.178132   -16.334824           0          440          665
    obj  4:     2.961445     1.638519     0.000000    -0.077256     3.384509    61.044972           0          225          741
Detected objects: 5
obj0: x=-0.12246578931808472, y=1.9556218385696411, z=0.0, v=0.0, snr=155
obj1: x=0.6123289465904236, y=1.861318826675415, z=0.0, v=0.0, snr=155
obj2: x=0.022266507148742676, y=0.17673492431640625, z=0.0, v=0.6180459260940552, snr=440
obj3: x=-0.05009964108467102, y=0.17094166576862335, z=0.0, v=0.6180459260940552, snr=440
obj4: x=2.9614453315734863, y=1.6385185718536377, z=0.0, v=-0.077257407617569, snr=225
```

Figure 2: An Output Frame showing Position, Velocity, and Number of Detected Objects

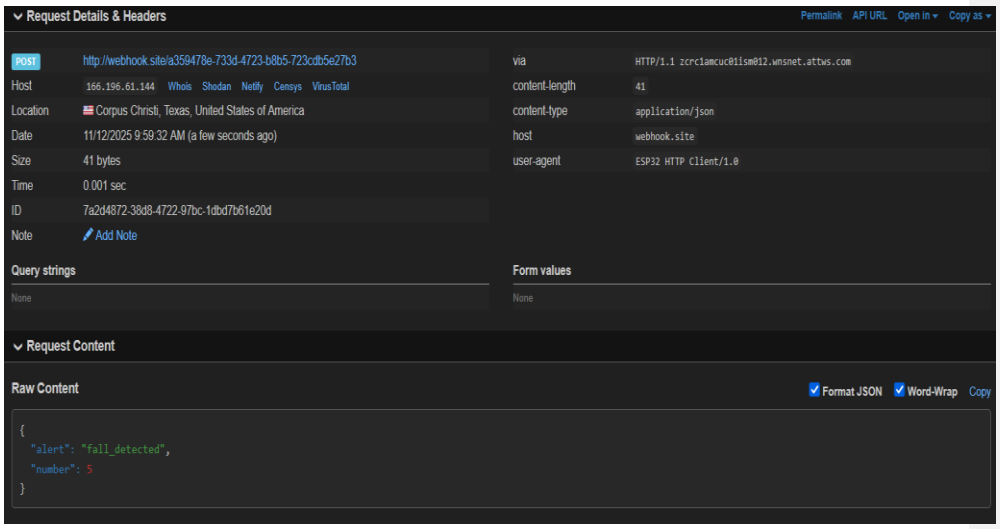


Figure 3: Webhook Showing the JSON file Contents

2.3 Subsystem Validation

The most important metrics to be validated for the microprocessor were to ensure that the data coming in from the TI radar board was taken in every frame at 20 FPS, and no frames were being dropped, along with frames being correctly defined. Figure 4 below shows an example of frames being dropped as a result of 0 objects being detected. Due to late complications with the initial design that utilized an ESP32-S3 microcontroller, the decision to switch to a Raspberry Pi 5 was made too late to properly validate subsystem using the Raspberry Pi. As such, the validation was done using an ESP32-S3 development board connected to the computer, and the computer was simultaneously connected with the TI radar board. In this setup, the computer acted as a middleman between the ESP32 and the radar board. Table 1 below details the validation plan.

Validation Criteria	Definition	Result
Correct Frames without Drops	>99% of Frames are Correctly Processed	Pass
Configuration Verified	Configuration Validated before Data is taken in	Pass
Wi-Fi Output Signals	>99% of JSON signals sent via Wi-Fi are correctly received	Pass

Table 1: Microprocessor Subsystem Validation Plan

```
************ Frame Fail, numDetObj = 0 ****************
headerStartIndex    = 0
totalPacketNumBytes = 192
platform            = b'000a1642'
frameNumber         = 19
timeCpuCycles       = 311082973
numDetObj           = 0
numTlv              = 2
subFrameNumber      = 0
************ Frame Fail, numDetObj = 0 ****************
headerStartIndex    = 0
totalPacketNumBytes = 192
platform            = b'000a1642'
frameNumber         = 19
timeCpuCycles       = 311082973
numDetObj           = 0
numTlv              = 2
subFrameNumber      = 0
```

Figure 4: Frame Failure

2.4 Subsystem Conclusion

The subsystem works as expected within the scope of the validation plan. The subsystem still needs to be fully moved over to the Raspberry Pi 5 before it can be fully integrated and run independently of an external computer. Beyond that, there is more work to be done in terms of matching the existing interfaces of each subsystem with each other.

3 Power Supply Subsystem Report

3.1 Subsystem Introduction

The power supply is designed to convert a volatile DC battery voltage into a regulated 5V output rail with minimum power loss. This regulated voltage is then used to power both the radar subsystem using a 2.1x5.5mm barrel jack and a Raspberry PI with a USB-C input. This battery is charged using a 24V DC input rail powering a battery charger IC providing constant current, constant voltage charging to the batteries. Both the voltage regulator and battery charger IC provide overvoltage and overcurrent protection to their output rails.

3.2 Battery Charger IC

Before the battery charging design could be chosen, the battery voltage and make-up had to be decided. Given the need for repeated charges and a desire to lower costs, lithium-ion batteries were chosen given the low cost and decent charge capacity. Lithium polymer batteries were considered for their higher current draws and faster charging speeds but were dropped due to higher costs and lower capacity. With the type of battery chosen, a specific model had to be selected, which in this case was the 1850 3.6V cell for its large use in industry allowing easy integration into the design. Because the input current draw through a buck regulator is proportional to the voltage division of the setup, lower currents through the battery can be achieved with higher voltages. This has a two-fold benefit for the final design; lower capacity batteries are needed to reach the time requirements, and a less complex charging setup can be used given most ICs are current limited rather than voltage limited. Based on these requirements, it was decided to use 4 batteries in series providing a nominal 14.4V voltage. Charging the battery of the subsystem is accomplished using a BQ24600VART. This is a battery charger IC presented by TI, which can charge multiple lithium-ion batteries in series while offering overvoltage and overcurrent protection for charging while also providing thermal shutoff protection for both the charger and the battery cells themself. Using the resources provided by Texas Instruments, the design in figure 5 could be created. For this, several key design choices needed to be made; charging current and the voltage limit of the device. Looking at the design, it can be seen that the charging current is set by the resistor divider powering the ISET pin, and the battery voltage by the VFB pin. Both of these values, along with the resistor divider connected to the thermistor, were chosen based on recommended values following the formulas presented in documentation.

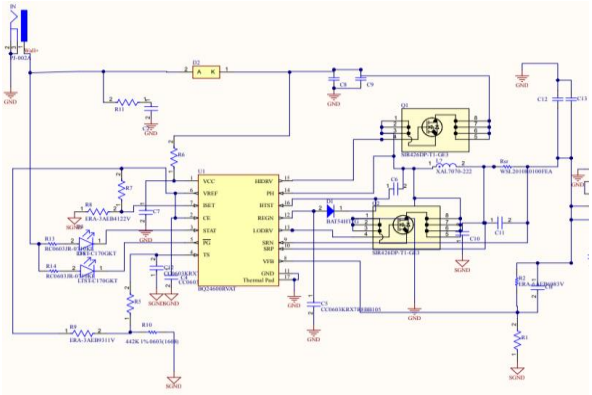


Figure 5: Buck Converter Current Simulation

3.3 Voltage Regulator

With the battery charger circuit decided, the voltage regulator could be designed. For this, there were two key considerations; output voltage/current, and the power used by the system. Given the large difference between the battery voltage and the given output, a buck converter was necessary in order to properly regulate voltage without excessive power loss. However, given the lack of voltage regulation present on the input for the Pi 5, ensuring stability of the output required an additional LDO on the output stage, or a different technology all-together. Based on this, it was decided to use integrated circuit implementation presented by Texas Instruments for their simplicity, ease of simulating, and high-performance metrics. Because of the high current requirements presented by the two systems, there were two options available for design of the voltage rail; using two separate regulators with 5V 5A capabilities or finding a 5V 10A regulator. Based on these needs the TPS56A37RPAR was chosen and a design could be created using the documentation provided by Texas Instruments, with an additional fuse attached before the barrel jack connections on the board for increased safety.

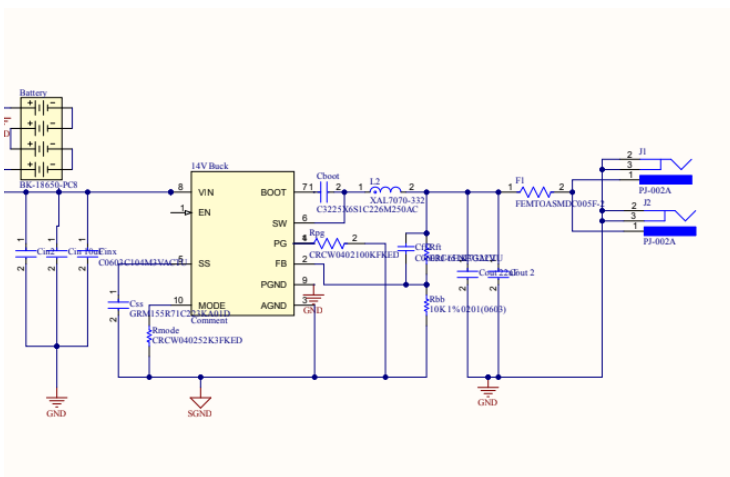


Figure 6: Power Supply Schematic

3.4 Subsystem Initial Validation

To validate the system's output, the design was first tested using OrCAD for its PSpice-for-Ti plug-ins. When testing the design, several concessions had to be made, as the battery charger IC was too complex for simulation, as stated by Texas Instruments documentation. Because of this, only the step-down converter could be tested and validated. To ensure proper operation of the voltage regulator, the voltage output was tested under two different conditions: varying input voltage and varying load resistance. Through this, the system could be validated across the input voltage range necessary

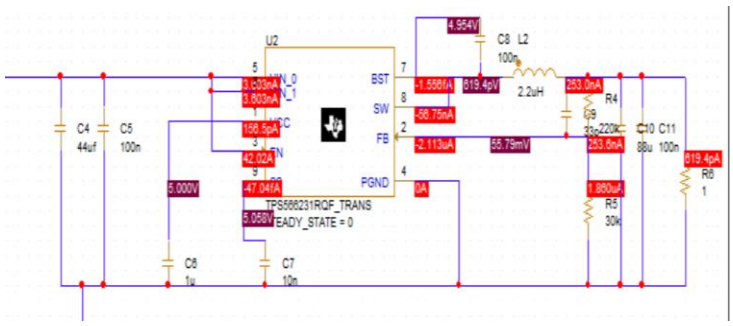


Figure 7: 5V Buck Converter Schematic

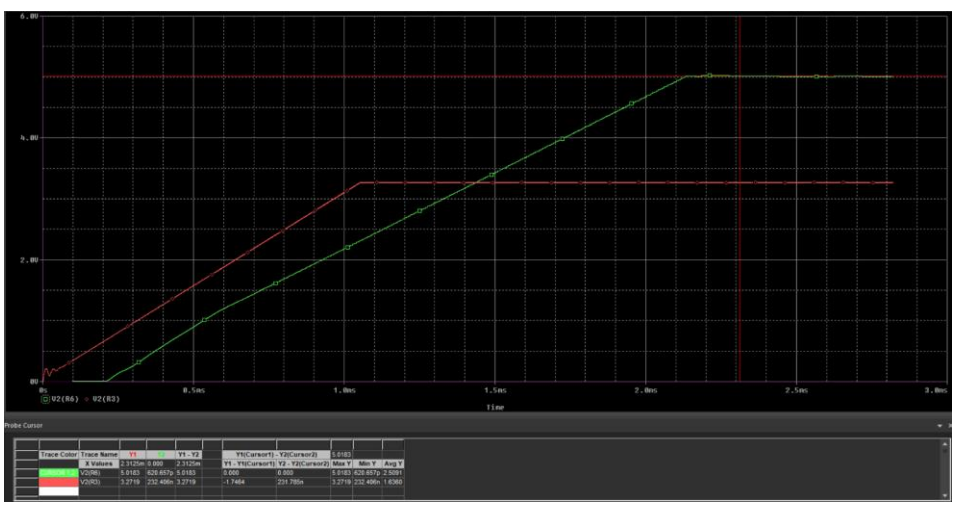


Figure 8: Buck Converter Voltage Simulation

Figure 9: Buck Converter Current Simulation

$$V_{out} = 0.6\left(1 + \frac{R_{Upper}}{R_{Lower}}\right)$$

3.5 Power Supply PCB

Using the data found above, a PCB design for the power supply could be made. For this, two main concerns needed to be addressed: ensuring the maximum current for charging and output could be safely achieved, and sufficiently low noise within the feedback circuit for both designs. Using Altium, a four-layer board was designed for both charging and output. While a higher cost was necessary with four layers, in order to properly route the components of the charging IC, it was necessary. Additionally, with this technique, the ground plane could be separated into two separate planes with a power ground and a signal ground. For components, SMD technology was utilized for all but the voltage regulator capacitor in the battery charger IC, given its low capacitance and high voltage requirements. The primary size used was 0603 for a balance between size and ease of soldering. However, up to 603 metric was used in cases where parasitic resistance and capacitance needed to be minimized. Because thermal heating was a big concern in the design, large ground pours were placed on both the signal and power planes. These could then be layered with vias, allowing for

thermal dissipation. This came with the added benefit of increased strength to the board, which was necessary given that the battery consists of 4 3.6V lithium-ion batteries, adding .44lbs that needs to be supported by the board alone. Finally, testing points were placed at the battery output and the voltage regulator output, with a test point at ground to allow voltage reading.

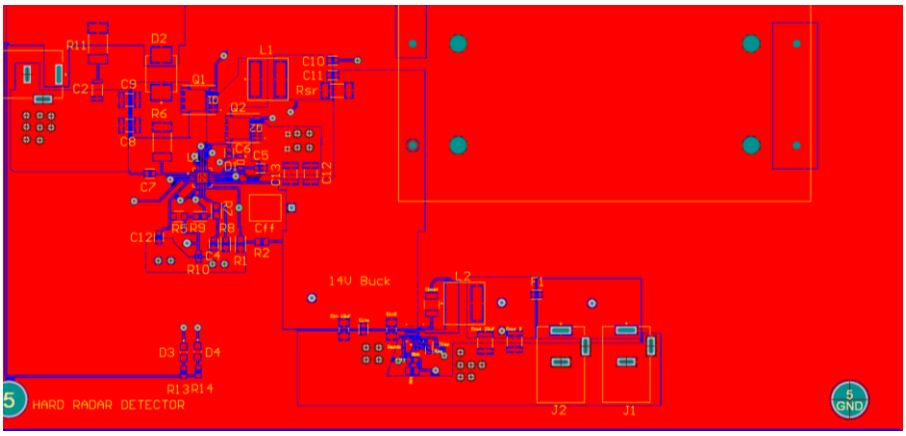


Figure 10: Power Supply Subsystem PCB Layout

3.6 Power Supply Assembly and Validation

Soldering the components to the board was using lead solder and a heat gun for most components. This was done in two primary steps. First, all SMD components besides the IC were attached to the board and tested for continuity and proper resistance and capacitance values at test points. Next, the BQ24600VART and TPS56A37RPAR were attached. From here, the testing of the power supply was accomplished in two main steps: first, the two separate systems were tested individually to ensure proper operation of both components; then, the system was charged while under a load to ensure both could be accomplished at the same time. For both the testing of the individual components and the entire board, voltages and currents were gradually stepped up to find faults in system design before detrimental damage could occur. However, in the validation and construction of the power supply, two boards were lost due to damage to the signal layer in testing, specifically around the Q2 MOSFET. This could be due to a variety of reasons; however, further iterations of the design would increase the spacing between the signal-layer ground plane and a widened signal trace.

Figure 11: Power Supply PCB

3.6.1 Battery Charger

Battery charging validation was conducted by charging depleted lithium batteries (3V) while measuring both the battery voltage and input current over time. Battery voltage was tested by measuring the difference between the voltage test pin and ground, while the current was found by dividing the voltage found by a current sense resistor by its resistance value. From this, the following transient response for the system could be found in figure 12. Using this, both the overvoltage and overcurrent protections for the system could be confirmed, with the current dropping from a maximum of 4A to 0 as the system charged. Additionally, while difficult to see due to high charging currents pushing the battery to voltage quickly; the initial voltage increase can be seen as well with the voltage output increasing with the constant current output. Finally, the system was tested while an electronic load was attached with no change to the output, however the charge status LED instead of turning off at full capacity began to display a charge fault periodically between charging due to the IC still pulling current above 0 to maintain the voltage.

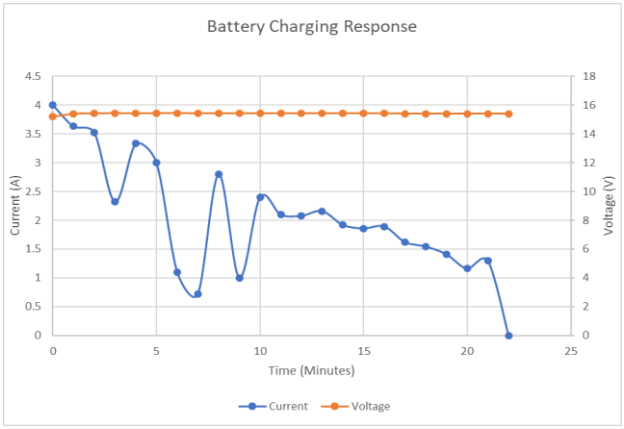


Figure 12: Battery Charging Voltage & Current Response

Battery life for the system was found by connecting the TI board and the MCU with a DC power supply connected where the batteries were. From this, the current pulled by the battery could be found thereby allowing calculations for runtime of the system to be made. Based on the results found in figure 13, it can be seen that an average current of 0.69A was pulled at 14.5V. Assuming that this is the average voltage is the between the maximum and minimum value, and that the current scaling is inversely proportional to the voltage, it can be calculated that for a 3300mAh battery the device could be run for a max of 4.5 hours. This is above the targeted 4 hours for the system.

Figure 13: Battery Charging Voltage & Current

3.6.2 Voltage Regulator

Proper voltage regulator output was confirmed using the same testing parameters as were used in OrCAD. For this, the system was tested under a load using a electronic load device, and both the input voltage for the regulator and the output current were swept as shown in figures 14 and 15. Looking at the output for these figures, it is seen that the device worked as intended across the voltage and current regime of the system with a slight reduction in max current for the system. Looking at the first figure for voltage response, it is noted that the device begins to operate at around 5.5V with a lower voltage possible at 4.5V. Past this, the device outputs a nominal 5V with very little variation in the output both in the average and when viewed using an oscilloscope as seen in figure 17. From this, the device can be confirmed to work under the expected voltage requirements presented by the battery in its charge cases. Overvoltage of the system was not tested given the battery voltage was already limited by the charging IC.

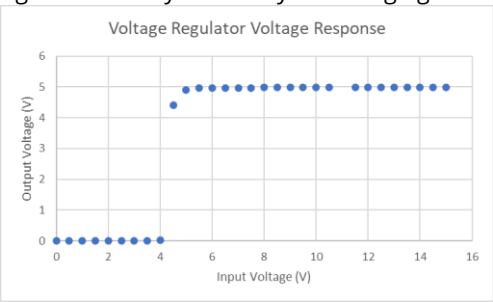


Figure 14: Battery Voltage

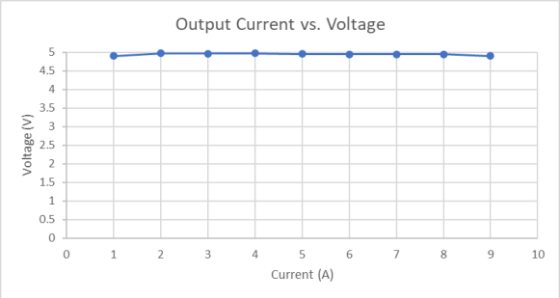


Figure 15: Battery Voltage vs Current

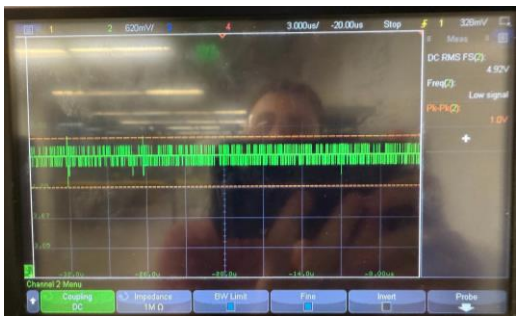


Figure 17: Voltage Output Transient Response

3.7 Power Supply Conclusion

Overall, the power supply worked as expected with relatively stable outputs in terms of voltage spikes and current draw. Integration with other subsystems is also easily achieved with standard barrel jack connectors used in the integration with other components. Some issues were found, specifically with trace width and a lack of internal connection of the signal and power ground, however these were either solved or nonissues assuming the system was operating under standard conditions. Further refinement of the design could lead to a more robust design, at the expense of time and cost, but are beyond the scope of the capstone requirements with these changes more important if scalability of the project became a necessary concern.

4 Machine Learning Subsystem Report

4.1 Subsystem Introduction

The machine learning subsystem is designed to reliably carry out the core of the fall detection task. The model used consists of a convolutional neural network trained on data collected from the radar board. The datasets were collected with the intention of capturing a wide range of what possible elderly fall injuries could look like, which includes many edge cases. Using standard neural network methods such as a validation & testing dataset, set aside to progressively evaluate the performance of the model across training iterations, the detection capability of the model was verified to function in close accord with the specifications required by the system.

4.2 Subsystem Details

Below is a block diagram of the ML subsystem

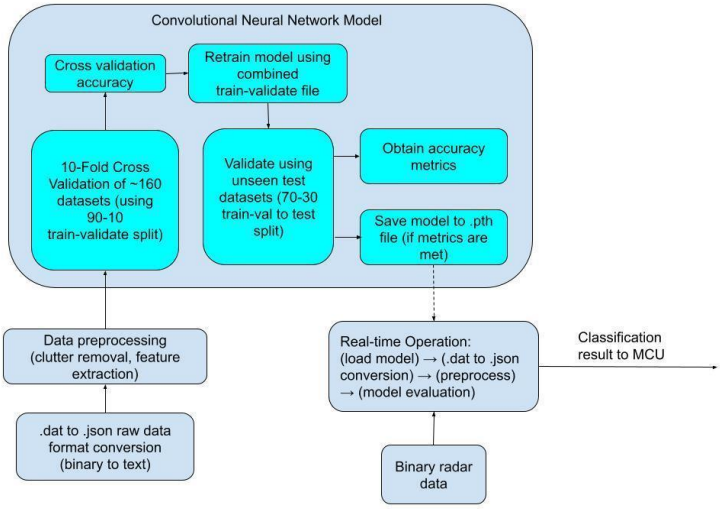


Figure 16: Machine Learning Subsystem Block Diagram

The design choices for the machine learning subsystem are as follows in the following sections. To preface, it was determined that a neural network would be an efficient architecture to perform the classification/detection task that the project requires, due to the high variance in what fall injuries could look like, rather than decision trees like random forest or linear regression methods.

4.2.1 Data Formatting

For fall detection, it was determined that tracking statistical features such as average vertical position, average doppler velocity, and downward acceleration of a radar cross section would be necessary for the model to distinguish between falls and non-falls. The data undergoes a process of clutter removal (for stationary objects and reflections) and moving averaging in order to consolidate a clear object for detection. In addition to this, the network utilizes the 256 range bin values provided by the radar's internal processing which is also featured in the model input. These values measure the signal strength in 256 increments of distance from the radar and therefore prove useful in tracking the motion pattern of an individual.

4.2.2 Neural Network Architecture

A convolutional neural network using a 1D convolution method was chosen as the specific architecture as it is well suited for sequential data like the radar's output. The network operates by sliding a kernel size of reasonable proportions (~1 second of data), performing a convolution filter across all frames of data. The model is flattened using max pooling, and then processed through a typical hidden layer network. The output of the network is a binary value indicating whether a fall (1) or non-fall (0) has occurred for a particular batch of data. Below is an example of training vs. validation loss for a typical training process:

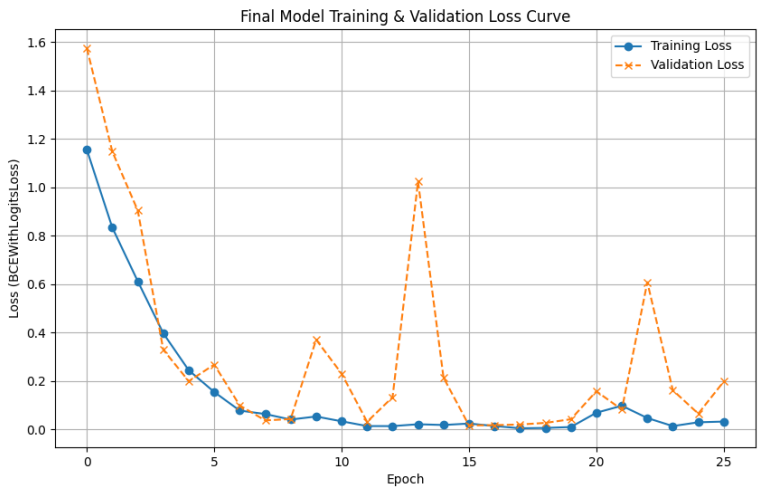


Figure 17: Training & Validation Loss Curves

4.2.3 K-Fold Cross Validation

A 10-fold cross validation method (which retrains the model across different permutations of training files) was implemented to ensure that the training data is sufficiently diverse for detection of edge cases. The method utilizes the combined train-validation set which comprises 70% of all recorded data and does a train-validation split of 90-10 for each fold. From here, the model is retrained 10 times, with the validation set moved around each time so that validation is applied to as many datasets as possible. Doing so allows the model parameters & output accuracy to be assessed as a whole,

rather than an arbitrary split which may or may not produce desirable results, depending on what files end up where.

Below is an example plot of validation accuracy across 10 folds:

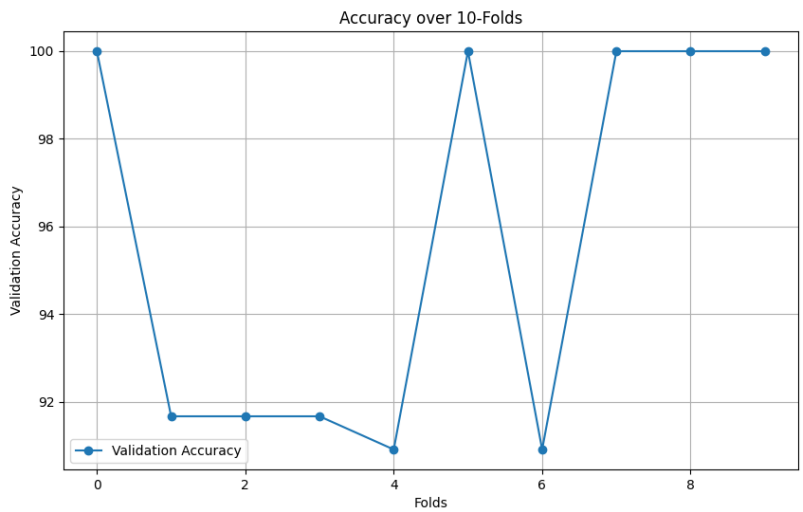


Figure 18: 10-Fold Cross Validation Accuracy

4.3 Subsystem Validation

The most important metrics for the subsystem are false positive and false negative rates. The false positive rate must be small, since false positives cost resources. The false negative rate must also be small, since missing a fall could have fatal consequences for the user. The subsystem is validated using a test dataset, which consists of 30% of the total data. This split was chosen to ensure that enough edge cases would be included in the test data, whilst also ensuring the model has enough training data to be robust.

Below are some performance criteria for the subsystem:

Validation Criteria	Definition	Progress (as of latest model)
False Positive Rate	A non-fall event which is classified as a fall event as a ratio of all non-fall cases	11.43%

False Negative Rate	A fall event which is classified as a non-fall event as a ratio of all fall cases	5.26%
Train & Test Dataset Size	The combined number of datasets used for model training and datasets for model simulation	249
Fall Positive Dataset	Number of datasets which serve as ground truth for falling scenario or '1'	80
Fall Negative Dataset	Number of datasets which serve as ground truth for non-falling scenario	169
Edge case handling	Model must account for motion patterns that do not conform to majority of fall or non-fall patterns	82
Confuser Rejection	Combined total of false positive and false negative classifications as a ratio of total test cases	8.22%
Noise handling	Model must account for noise that may be present in real-life residences	Noise injection present in training process
Detection delay	Model must be able to completely process any falling motion data within a reliable timeframe	6.75 s
Computation Speed	Real-time data must be converted to. json, preprocessed, and classified before the processing of the following data batch. A standard data batch will be 400 frames, so a dataset of this length will be used to set the benchmark.	0.766 s

Table 2: Machine Learning Subsystem Performance Criteria

A screenshot of terminal output to validate that the model can perform at the metrics specified above:

```
CROSS-VALIDATION SUMMARY

Training Accuracy: 100.00% ± 0.00%
Validation Accuracy: 97.09% ± 4.45%

Aggregated Confusion Matrix (all validation folds):
[[65  2]
 [ 1 38]]
True Negatives:  65 (61.3%)
False Positives: 2 (1.9%) - false alarms
False Negatives: 1 (0.9%) - MISSED FALLS
True Positives:  38 (35.8%)
Fall Recall:     97.4% (sensitivity)
Fall Precision:  95.0%

FINAL TEST SET EVALUATION

Retraining on full train+val set...
Early stopping at epoch 26

Test Accuracy: 98.28%
Test Confusion Matrix:
[[37  0]
 [ 1 20]]
```

Figure 19: Terminal Output for Model Training Run

The false positive, false negative rates, and computation speed are within reasonable margins for a reliable operation of the detection system.

4.4 Subsystem Conclusion

The most critical components of the subsystem are shown to be working correctly. The neural network is able to learn from given datasets and classify new data at a high rate of accuracy and do so within a reasonable timeframe.

5 App & Interface Subsystem Report

5.1 Subsystem Introduction

The mobile application subsystem serves as the real-time user interface and monitoring layer of the Human Activity Radar Detector (HARD) system. Its primary role is to receive, process, and present fall detection events generated at the edge device, which consists of a Raspberry Pi and mmWave radar sensor. The application is designed within a distributed system architecture, where detection results are transmitted from the edge device to a cloud backend and then propagated to the user through real-time data streams and push notifications.

This subsystem enables immediate alert delivery, user interaction through acknowledgment mechanisms, and visualization of both real-time and historical data. It also provides system-level visibility into device operation, allowing users to monitor not only detected events but also overall system health. The application is implemented using Flutter and integrates with Firebase services, including Authentication, Cloud Firestore, and Firebase Cloud Messaging, to achieve secure access, scalable data handling, and low-latency communication.

5.2 Subsystem Details

5.2.1 Mobile Application Infrastructure

The HARD mobile application operates as a client within a cloud-connected architecture, leveraging Firebase as its backend platform. Firebase was selected due to its ability to support real-time data synchronization, scalable NoSQL storage, and integrated authentication and messaging services. These features allow the system to

maintain continuous synchronization between the edge device, backend, and user interface.

User authentication is handled through Firebase Authentication, ensuring that only authorized users can access system data. Alert data and device state information are stored in Cloud Firestore, a document-based NoSQL database that supports flexible schema design and real-time updates. The application subscribes to Firestore collections using persistent connections, allowing it to receive updates immediately when new data is written. This eliminates the need for periodic polling and reduces latency.

Push notifications are handled through Firebase Cloud Messaging, which enables alerts to be delivered to users even when the application is running in the background. Together, these components form a responsive and scalable infrastructure that supports real-time monitoring and user interaction.

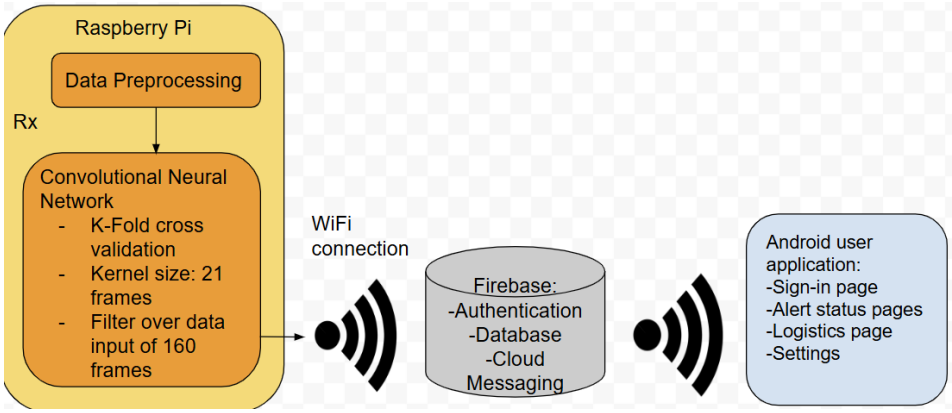


Figure 20: HARD Mobile Subsystem Data Flow

5.2.2 Data Model and Storage Design

The HARD system utilizes a NoSQL document-based data model implemented in Cloud Firestore to store alert and device information. Each alert is represented as a document containing key fields such as the timestamp of detection, device identifier or room location, classification result, confidence score, and acknowledgment status. This

flexible schema allows for easy expansion and modification without requiring rigid database restructuring.

Field	Meaning
timestamp	Time the alert was created
type	Type of detection event
room	Optional location information
confidence	Optional confidence value from the AI detection
acknowledged	Indicates whether the alert has been reviewed
acknowledgedBy	User who acknowledged the alert
acknowledgedAt	Timestamp of the acknowledgement

Table 3: App User Field Statistics

The database is organized into collections that support efficient querying and filtering. Alerts can be categorized based on acknowledgment status, enabling the application to separate active alerts from historical records. This structure also supports aggregation and analytics, which are used to generate insights such as alert frequency and room activity trends within the logistics interface.

5.2.3 Software Architecture Stack

The software stack for the mobile subsystem combines Flutter's UI framework with several Firebase and third-party packages. Flutter handles all user interface elements, navigation between screens, and the overall styling of the app. Firebase's libraries provide the backend functionality—initializing the app, validating user credentials, reading and writing Firestore data, and receiving cloud messages.

Additional packages support specialized features. The flutter_local_notifications package allows the app to display Android notifications when a fall alert arrives while the app is open. The intl package is used to format timestamps into readable date/time strings. The fl_chart package powers the bar charts and summary graphics in the analytics tab.

Together, these components form a lightweight but capable architecture that can grow as the HARD project expands.

5.2.4 Application Interface and Functional Design

The mobile app is organized around three main tabs that separate alerts by their status and provide additional context for system behavior.

Unacknowledged Alerts:
This tab displays all active fall alerts that have not yet been reviewed by a user. The information updates automatically as new data is written to Firestore. A small red badge on the tab icon indicates how many unacknowledged alerts are currently waiting.

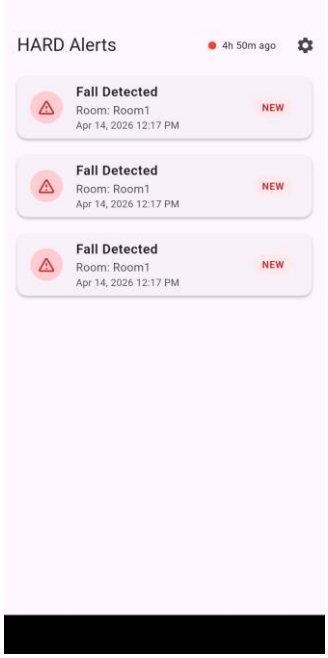


Figure 21: Unacknowledged Alerts Tab

Acknowledged Alerts:
Past alerts that have already been addressed are shown here. This gives users a simple history of events and allows quick verification that an alert was acknowledged correctly.

Figure 22: Acknowledged Alerts Tab

Analytics Tab:
This tab summarizes system activity using charts and room-by-room statistics. The app calculates totals, identifies the most active rooms, and estimates average response times by comparing alert timestamps to acknowledgement times. A bar chart provides a visual overview of alert frequency, and users can tap into individual rooms to see more detailed information.

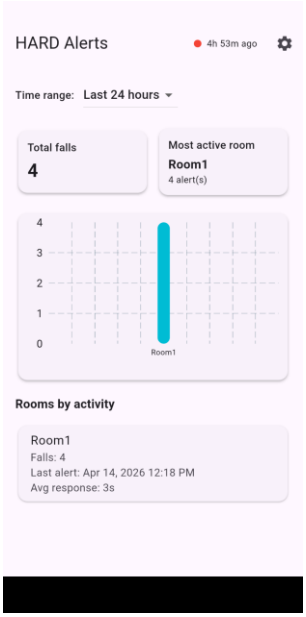


Figure 23: Logistics/Analytics Page

In addition to the three main tabs, the app includes a login page for user authentication, a detailed alert page that displays all information for a single alert, and a settings page accessible from the top-right corner of the home screen. The app also supports deep linking from notifications—when a user taps a notification, they are taken directly to the relevant alert detail screen.

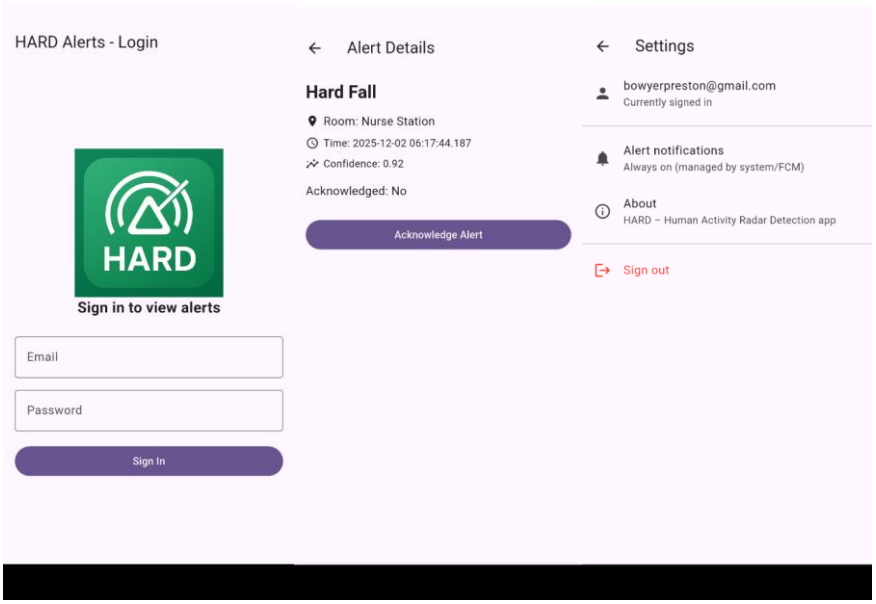


Figure 24: Login, Alert Details, and Settings Pages

5.2.5 System Data Flow Pipeline

The HARD system operates through a multi-stage data pipeline that connects radar sensing hardware to the end-user application. Motion data is continuously captured by the mmWave radar and transmitted to the Raspberry Pi, where it undergoes preprocessing and machine learning inference. The model classifies the data as either a fall or non-fall event, generating both a prediction and an associated confidence score.

When a fall event is detected, the Raspberry Pi sends a structured HTTP request containing the event data to the cloud backend. This request updates the Cloud Firestore database by creating a new alert document. The database write acts as an event trigger, making the new alert immediately available to any subscribed clients.

Simultaneously, Firebase Cloud Messaging generates a push notification to inform the user of the detected event. The mobile application, which maintains an active real-time listener on the database, receives the update and automatically refreshes the user interface. This pipeline enables low-latency communication from detection to user notification without requiring manual refresh or polling mechanisms.

5.2.6 System Health Monitoring (Heartbeat)

In addition to event detection, the system includes a heartbeat mechanism to monitor the operational status of the edge device. The Raspberry Pi periodically transmits a heartbeat signal to the backend via HTTP requests at fixed intervals. These requests update device-specific fields in the Firestore database, including timestamps indicating the most recent communication.

The mobile application subscribes to this device data using the same real-time listener architecture used for alerts. By comparing the current time to the last recorded heartbeat, the application determines whether the device is online or offline. This allows users to quickly identify system failures, connectivity issues, or power interruptions.

The heartbeat mechanism enhances system reliability by ensuring that users are informed not only when events occur, but also when the system is no longer functioning as expected.

5.3 Subsystem Validation

The End-User Application subsystem was validated through a series of functional and performance-based test scenarios designed to evaluate real-time operation, data integrity, user interaction, and system reliability. Testing was conducted using both controlled backend inputs and live system interaction to ensure accurate representation of real-world operating conditions.

Authentication functionality was validated by testing multiple valid and invalid credential combinations using Firebase Authentication. Results confirmed that all valid login attempts were successfully authenticated, while invalid credentials were consistently rejected with appropriate error messaging. This demonstrated reliable access control and ensured that only authorized users could interact with system data.

Real-time alert retrieval was verified by inserting alert documents directly into the Firestore database. The application successfully updated the user interface without requiring manual refresh, confirming correct implementation of Firestore stream listeners and event-driven updates. Alerts consistently populated the appropriate interface views, including Unacknowledged and Acknowledged tabs, demonstrating proper filtering and categorization logic.

Notification delivery was tested under both foreground and background application states. When the application was active, alerts triggered immediate in-app notifications and correctly displayed detailed alert information upon user interaction. When the

application was minimized or closed, Firebase Cloud Messaging successfully delivered system-level notifications, and tapping these notifications routed users to the appropriate alert detail screen. These results confirmed reliable push notification behavior across application states.

Alert acknowledgment functionality was validated by updating alert states within the application and verifying corresponding changes in the Firestore database. Fields including acknowledgment status, acknowledging user, and timestamp were correctly updated, and the application interface reflected these changes in real time by transitioning alerts between active and acknowledged views.

System performance under high load was evaluated by injecting a batch of 100 alerts into the Firestore database within a short time interval. The application successfully processed and displayed all alerts without observable user interface lag or data loss. This test confirmed that the subsystem can handle elevated event activity while maintaining responsiveness and data consistency.

Notification reliability and latency were also evaluated. A batch test involving 200 alert notifications confirmed that all notifications were delivered successfully with no missed events. Individual notification timing tests showed that alerts were consistently delivered to the user device within approximately five seconds of database update, demonstrating acceptable end-to-end latency within system requirements.

The logistics and analytics interface was validated by generating alerts across multiple rooms and time periods. The application correctly calculated and displayed total alert counts, room-based activity metrics, and graphical trends. All displayed values were verified against Firestore data, confirming the accuracy of aggregation and visualization logic.

Overall, validation results demonstrate that the End-User Application subsystem operates reliably under both normal and high-load conditions. The subsystem maintains real-time synchronization with backend data, ensures secure user access, and provides accurate and responsive visualization of system events. These results confirm that the application fulfills its role within the HARD system as a dependable and scalable user interface.

Test	Feature	Description	Condition	Expected Result	Result
T1	Authentication	Log in using valid and invalid credentials	Test user accounts in Firebase Auth	Valid logins succeed; invalid logins return appropriate error messages.	Pass
T2	Alert Retrieval	Confirm alerts appear in real time	Insert sample documents into alerts collection	New alerts populate Unacknowledged tab without refresh.	Pass
T3	Notifications	Receive alert while app is in the foreground	App open; send FCM test alert	Alert triggers a local notification; tapping opens correct detail screen.	Pass
T4	Notifications	Receive alert while app is backgrounded or closed	App minimized or killed	System notification delivered; tapping navigates to alert detail screen.	Pass
T5	Acknowledgment	Acknowledge alert and verify Firestore update	Existing alert in database	Fields acknowledged, acknowledged By, and acknowledged At update correctly.	Pass
T6	Analytics	Validate counts, room activity, and response time calculations	Alerts generated across multiple rooms	Analytics tab displays accurate totals, timestamps, and average response time.	Pass
T7	Logout	Test sign-out functionality	User authenticated	User redirected to login screen; main app inaccessible until re-authenticated.	Pass
T8	Notification Reliability	Send 200 alerts and verify delivery consistency	Batch of 200 FCM alerts sent to device	Device receives all 200 notifications with correct routing behavior (0 missed).	Pass
T9	Notification latency	Measure alert delivery time from Firestore to device	Alerts sent individually while timing with stopwatch	Each notification arrives within 5 seconds of dispatch.	Pass
T10	System Tolerance	Validate app performance under high alert volume	Inject 100 alerts into Firestore in rapid succession	Each notification arrives within 5 seconds of dispatch.	Pass
T11	User Authentication	Verify correct acceptance and rejection of login attempts	Test multiple valid and invalid credential combinations	All valid credentials successfully authenticated; all invalid credentials rejected with appropriate error messaging (100% accuracy)	Pass
T12	Alert Display	Validate complete alert information is correctly rendered in UI	Generate alerts with full dataset (confidence, location, timestamp, acknowledgment state)	All alert fields displayed accurately in both list view and detail view	Pass

T13	Logistics / Historical Analytics Validation	Verify correctness of storical data aggregation nd visualization	Generate alerts across multiple rooms and time periods	Logistics tab correctly displays total alerts, room activity breakdown, and graphical trends; values match Firestore data	Pass
T14	Alert Management	Validate correct separation and updating of alert states	Acknowledge alerts within app	Alerts move from Unacknowledged to Acknowledged tab; Firestore fields update correctly and UI reflects changes immediately	Pass

Table 4: Mobile Subsystem Validation Test

Note: Highlighted tests are from ECEN 403, while unhighlighted are test conducted in ECEN 404 under full system testing/validation

5.4 Subsystem Conclusion

The End-User Application subsystem successfully meets its design objectives as the primary interface between the HARD detection system and the user. The application demonstrates reliable real-time alert delivery, accurate data visualization, and effective user interaction through acknowledgment and authentication features. Integration with Firebase services enabled a scalable and event-driven architecture, allowing alerts to propagate from the edge device to the user interface with minimal latency.

Validation testing confirmed that the application maintains stable performance under high alert volumes, correctly processes user authentication requests, and preserves data integrity across all displayed alert fields. The logistics interface further extends system functionality by providing meaningful historical insights, including alert trends and room-based activity analysis, without degrading application responsiveness.

Overall, the subsystem demonstrates strong reliability, responsiveness, and usability within a distributed system environment. Its ability to handle real-time data updates, maintain synchronization with backend services, and support user-driven interaction ensures that it effectively fulfills its role within the complete HARD system.

Radar Human Activity Detection

Jean Jacques Chirat
Preston Bowyer
William Jachimiak
Aiden George

FINAL SYSTEM REPORT

GITHUB REPOSITORY: HTTPS://GITHUB.COM/PRESTON11-TAMU/ECEN404-2026SP-TEAM-61-HARD

REVISION – 1
27 April 2026

FINAL SYSTEM REPORT

FOR

Human Activity Radar Detector

PREPARED BY:

_____

Author Date

APPROVED BY:

_____

Project Leader Date

_____

John Lusher, P.E. Date

_____

T/A Date

Change Record

Rev.	Date	Originator	Approvals	Description
1	4/26/2026	Human Activity Radar Detector Team		Initial Release for 404 Final Report

Table of Contents

1. Introduction 84
2. Subsystem Changes 84
2.2 Machine Learning Subsystem 84
2.3 Power / PCB Subsystem 85
2.4 End-User Application Subsystem 85
3. System Integration 86
3.1 Hardware-Hardware Integration (PCB-SBC/Radar) 86
3.2 Hardware-Software Integration (Radar-SBC-ML) 86
3.3 Cloud Integration (SBC-Cloud, Cloud-App) 87
3.4 Radar to SBC to ML to App pipeline 87
3.5 Physical Model Design 88
4. System Validation 88
4.1 Validation Approach 88
4.2 Full System Validation 89
5. System Performance 90
6. Challenges and Solutions 90
7. Design Strengths and Areas of Improvement 92
8. Conclusion 92

1. Introduction

The Human Activity Radar Detector is intended to be used in residential areas to serve as a means of passive fall detection monitoring. The system is composed of four subsystems: Radar/SBC, Machine Learning, Power, and the App. Throughout 404, emphasis was placed on integration of the subsystems into one complete system, particularly the integration of SBC/Radar with the ML and the App and then powering that integrated system. The sections below will detail the changes that were made to the subsystems in 404, the integration of the subsystems, and the validation and testing of the integrated system.

2. Subsystem Changes

2.1 Radar + SBC Subsystem

The code that had previously been written for the ESP32 (in C++) had to be migrated to the Raspberry Pi (in Python). This was due to previously discussed issues with the computing power of the ESP32 and interfacing issues with the ESP32 and the TI1642 Radar. Beyond the emigration, there were a number of other key changes to the subsystem that had to be made. The first was to add a variable sliding queue to store in frames from the radar and feed it into the ML subsystem (with the size and overlap being variable). Additionally, as part of integration, a radar frame data structure was defined to more easily allow for the data coming in to be read from.

To make all of these systems work in tandem, a data pipeline was created that utilized threading to run all of the processes at the same time (data and Wi-Fi), with the ML only getting called every x second (where x was the variable overlap time). The last two features that were added to the subsystem were a pulse that was sent out to the app that told the app the SBC was on, and the ability for main.py to run on boot and restart on the Pi.

2.2 Machine Learning Subsystem

The machine learning subsystem, which is responsible for training and developing the neural network used to perform inference on incoming radar signals, had to be reworked upon testing in a real-world deployment of the full system. The features which comprise the input vector into the model, which are determined in the preprocessing script, had to be modified to massively decrease the false positive rate which the system exhibited in the initial testing. The changes involved the removal of the radial velocity,

the inclusion of the y-coordinate (the horizontal distance from the radar), and an emphasis placed on the SNR measurements in the form of calculating the relative RCS (radar cross section) using these available measurements. Within the preprocessing script in the course of deployment, feature smoothing algorithms using Savitsky-Golay filters were also used in an attempt to reduce the noisy nature of radar signal reflections.

Additional test cases also needed to be collected in order to cover the most essential edge cases specified by the system requirements. These include sitting down, standing up, jumping, and walking test cases. Overall, the additions to the training dataset number at about 100, to an overall training dataset of 249 samples.

2.3 **Power / PCB Subsystem**

Between the first and second semester, the power supply PCB was drastically changed to facilitate the change from an ESP-32 MCU to a Raspberry PI-5. This new design changed the battery voltage from an original 7.7V to 14.4V for increased duration while maintaining the same battery charger architecture with changes to resistances. The voltage regulator architecture was also modified with the move to a single TPS56A37RPAR for regulation purposes. This modified architecture is what can be seen in the preceding subsystem report section with the old design removed from the report. Because of this, further refinements could be made to the PCB in Altium with increased usage of voltage and ground planes and the increase in trace thickness for high current subsections.

2.4 **End-User Application Subsystem**

The End-User Application subsystem was further refined this semester to improve reliability, system integration, and overall usability. While core functionality such as real-time alert display, Firebase integration, and push notifications was established in the previous semester, this semester focused on strengthening these features and validating their performance under realistic system conditions.

The primary new functionality introduced was a device monitoring capability through a heartbeat mechanism. The Raspberry Pi periodically sends HTTP requests to the backend, which updates a Firestore devices collection with the system's last active timestamp. The application uses this information to determine device connectivity in real time, allowing users to view whether the system is online or offline. This addition significantly improved system observability and provided visibility into system health that was not previously available.

In addition to this new feature, existing components were improved and more tightly integrated into the full system pipeline. Real-time data handling using Firestore listeners was validated and optimized to ensure consistent UI updates without manual refresh. Push notification behavior was refined to ensure reliable delivery in both foreground and background states, with correct navigation to alert details upon user interaction.
The data model was also enhanced to better support system validation and user interaction. Alert documents were structured to consistently include confidence score, timestamp, location, and acknowledgment metadata, enabling accurate display and traceability of events. The logistics and analytics interface were further developed and validated to ensure correct aggregation of alert data across rooms and time periods.

3. System Integration

3.1 **Hardware-Hardware Integration (PCB-SBC/Radar)**

System integration between the power supply and the two subsystem boards was accomplished through the use of 2.1 x 5.5mm barrel jack connectors on the power PCB board. With these, a male-to-male cable could be connected for the radar board and a barrel jack to USB-C cable for powering the Raspberry Pi-5. These connectors were chosen for their simplicity and ease of integration between the subsystems. After power supply validation was completed, the systems were connected and powered using the internal battery supply with no issues for either board with both maintaining power for extended periods of time.
One issue found with the design, however, was the lack of a minimum voltage cut-off for the voltage regulator. Because of this, it was possible for the system to drain batteries well below their expected minimum voltage level. Because the PCB design left the EN pin for the voltage regulator floating, it was impossible to modify this behavior without a complete reorder of the PCB costing hundreds of dollars. With future design, this could be solved by adding a high resistance voltage divider between the EN pin and the battery voltage. For this reason, testing the battery voltage periodically during system testing was important to conserve battery health.

3.2 **Hardware-Software Integration (Radar-SBC-ML)**

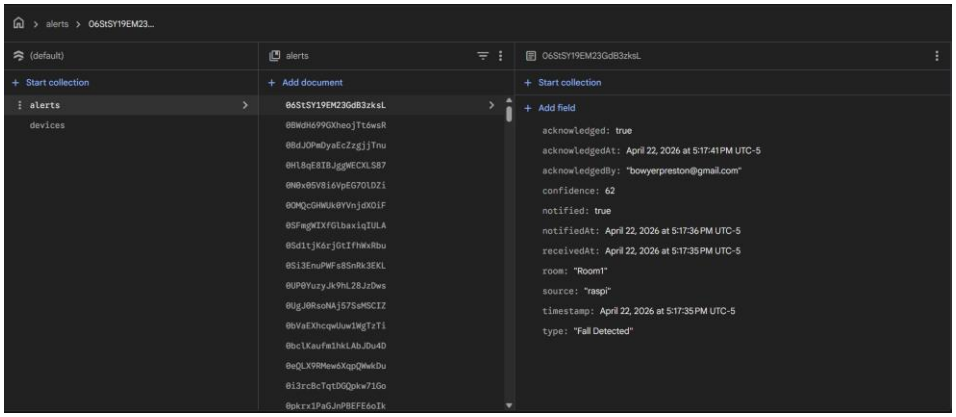
The SBC configures the Radar board, and once configured, the SBC starts to take data from the radar into a sliding queue. The data is of the form shown below:


```
=268)]), RadarFrame(frame_id=740, num_objects=3, objects=[DetectedObject(x=-0.0890660285949707, y=0.706939697265625, velocity=0.0, snr=253), DetectedObject(x=0.1168991625308990 5, y=1.2414326667785645, velocity=0.0, snr=341), DetectedObject(x=0.1836986839771270 8, y=1.9508227109909058, velocity=0.0, snr=262)]), RadarFrame(frame_id=741, num_objects=3, objects=[DetectedObject(x=-0
.0890660285949707, y=0.706939697265625, velocity=0.0, snr=254), DetectedObject(x=0.1168991625308990 5, y=1.2414326667785645, velocity=0.0, snr=339), DetectedObject(x=0.1836986839771270 8, y=1.
9508227109909058, velocity=0.0, snr=259)]), RadarFrame(frame_id=742, num_objects=3, objects=[DetectedObject(x=-0.0890660285949707, y=0.706939697265625, velocity=0.0, snr=252), DetectedObject
(x=0.1168991625308990 5, y=1.2414326667785645, velocity=0.0, snr=340), DetectedObject(x=0.1836986839771270 8, y=1.9508227109909058, velocity=0.0, snr=262)])]
```

This sliding queue constantly adjusts what data is within it according to the initialization values and then this data is run on the ML every x seconds, where x is the number of seconds of overlap in the sliding queue (5 seconds for the ends result). These radar frame data structures are then preprocessed in the Machine Learning algorithm, and the algorithm uses the processed data to produce a boolean output if there is a fall or not.

3.3 **Cloud Integration (SBC-Cloud, Cloud-App)**

The system utilizes cloud-mediated architecture to decouple edge processing from user interaction. The Raspberry Pi communicates with the backend using HTTP POST requests to Cloud Functions, which validate incoming data and write structured entries into the Firestore database. This includes both alert data and periodic heartbeat signals used for device monitoring.



The mobile application connects to Firestore using real-time listeners, enabling immediate propagation of updates from the backend to the user interface. This event-driven model ensures low-latency communication without requiring direct device-to-app connectivity. The integration supports scalability, as multiple devices and users can interact with the system simultaneously without additional coordination logic.

3.4 **Radar to SBC to ML to App pipeline**

Following the fall, the radar sends the data to the Raspberry Pi where it is stored in a sliding queue and sent to the Machine Learning algorithm which detects the fall before alert data is transmitted to the cloud backend via HTTP. Once validated, the alert is stored in Firestore, which triggers downstream processes including push notifications

and real-time UI updates. The mobile application receives these updates through Firestore streams and Firebase Cloud Messaging, allowing the user to view, acknowledge, and analyze alerts with minimal delay. This completes the full system pipeline from physical event detection to user response.

3.5 **Physical Model Design**

For the integration of all design components, a 3D printable enclosure was designed in SolidWorks. For this, the enclosure was designed with two sliding trays for holding the power PCB, and the other for mounting both the radar board and the PI-5. The boards were then directly screwed into the sliding trays using standard hardware, and the various components connected using their cables. This enclosure could then be directly mounted on a wall using Velcro adhesion strips due to the light weight of the total system. The power PCB could then be supplied with power from a hole in the bottom where a standard barrel jack connector is recessed. Further improvements to this design could be taken in the future, with a revamped design allowing easy removal of the power PCB. However, for proof-of-concept validation, it provided a simple and cheap solution to the problem.

4. System Validation

4.1 **Validation Approach**

Our approach to full-system validation involved the physical assembly of all subsystems, including the PLA system housing, in order to achieve accurate validation in the environment the system is expected to operate in. In this approach, the system

performance validation was done in a manner largely similar to the approach used for the collection of training data. The data pipeline within the SBC is standardized to evaluate over windows of 20 seconds each, with a 5-second-wide stride, and this structure is reflected in our validation process. Given that each window shifts once per 5 seconds, we reasonably expected that each fall or non-fall event will appear in the data pipeline at most 4 times before it is discarded. Therefore, our validation testing allowed for at most 4 iterations of inferencing before a specific event could be definitively recorded as a fall/non-fall.

For the environmental considerations of our validation approach, it was a stated objective to perform testing in a variety of settings in order to ensure that the system functioned up to standards in the specified environments. In line with this requirement, testing was performed in hallways and enclosed rooms, with dimensions typical for a residential home. However, it quickly became clear that a radar-based detection system would need to avoid areas containing large, metal objects/industrial equipment, as these things inject major reflectional noise in the raw data of the radar. The areas which have been written off for system operation are places with large electronic equipment such as kitchens, garages, as well as open areas in general.

Lastly, for the physical system validation, such as the battery life, the trial was done in a rough manner, as precise time measurements over the course of hours and accounting for when the system was running and the amount of current draw at each point in time could not be done. Thus, the battery life was calculated using voltage measurements over a short-period and extrapolated.

4.2 **Full System Validation**

In order to perform full system validation, the physically assembled (including physical enclosure) system was deployed in varied areas, and tested by performing the same type of physical motions in the radar's field-of-view. The procedure is explained in detail above in the validation approach section. For example, by acting out a basic fall in front of the radar, by walking perpendicular, parallel to the radar, etc. The timestamp of the fall was recorded, and around a maximum of 20 seconds after each event was spent observing the output to make sure that the algorithm did not throw a wrong prediction in the 4 possible inferences on the window.

For the system validation, 73 formal trials were performed, along with some informal trials and testing during debugging attempts (which were not recorded). Of these 73 trials, 38 trials were falls and 35 trials were non-falls. Fall test cases were scrutinized

more heavily since the design philosophy regards false negatives as more costly than false positives. The results for the validation trials can be seen in the section below. Lastly, using the aforementioned battery life calculations and over the course of long periods of validation, the battery life was determined to meet the validation requirements.

5. System Performance

Field	**Validation Goal**	**Result**
False Positive Rate	95%	89%
False Negative Rate	95%	95%
Overall Accuracy	95%	94.5%
Average Response Time (s)	<20	6.75
Maximum Response Time (s)	20	20
Maximum Detection Range (m)	>5	9.0
Maximum Battery Life (hr)	>4	4.5

6. Challenges and Solutions

Throughout the development of the HARD system, multiple challenges were encountered across hardware, signal processing, machine learning, and system integration, requiring iterative design changes to achieve stable and reliable operation.

Thermal management of the embedded processing unit was identified as an early issue, as sustained operation of the Raspberry Pi under continuous data processing led to elevated temperatures. To address this, a heat sink was added to improve thermal dissipation and ensure consistent performance during extended runtime testing.

Signal processing and data pipeline stability also required multiple refinements. The sliding window queue used for feeding the machine learning model was adjusted

several times to balance responsiveness and robustness. It was observed that shorter windows were more susceptible to noise disturbances, particularly in real-world environments, leading to unstable predictions. Increasing the window length improved stability by providing more consistent temporal context to the model, reducing sensitivity to transient noise.

The machine learning subsystem underwent several retraining cycles to improve detection accuracy and adapt to observed real-world conditions. Variability in testing environments, including differences in room geometry and external interference, introduced inconsistencies in model performance. In particular, metallic surfaces within lab environments caused radar signal reflections, leading to noisy or distorted input data. These environmental effects required both data processing adjustments and model retraining to improve robustness.

Hardware reliability challenges were encountered within the power subsystem. The custom power supply PCB required multiple revisions due to component-level vulnerabilities. Specifically, the signal trace connected to the MOSFET gate was found to be prone to physical damage, necessitating redesign for improved durability. Additionally, parasitic resistance and capacitance effects within the circuit impacted performance, requiring modification of VBF return resistor values to maintain stable operation within the desired electrical characteristics.

At the system level, these challenges highlighted the importance of co-design across subsystems. Changes in hardware directly impacted signal quality, which in turn affected machine learning performance and overall system reliability. Through iterative testing and refinement, the system was improved to achieve stable operation across a range of conditions, demonstrating resilience to both environmental variability and hardware constraints.

7. Design Strengths and Areas of Improvement

From the experience of the validation testing, the major pitfalls of the current detection system is due to the false positive & false negative rate of the inference algorithm. The primary avenue of improving these metrics is simply to collect much more training data. For the convolutional neural network architecture upon which this system is based, the dataset threshold for a reliable output is higher than what has been collected for this particular implementation. Data collection is additionally limited in that no existing datasets exist for the IWR1642 in the task of fall detection. In light of this fact, it is much more understandable why the behavior of a model produced by a particular training run may swing unpredictably from another model produced with the same exact training process.

Another factor of improvement may be the design choices made in the preprocessing script. Visual inspection of the preprocessing will show that the plotted coordinate data is not as clean as would be preferred for a neural network which already suffers from a shortage of training datasets. For example, residual noise is still very much present although in much reduced quantity in the output of the data processing pipeline. These aberrations, even if sparse, cause the model input to skew towards the noise and produce swings that compound proportionally over the course of the training epochs. In short, many improvements can still be made by observing recorded sequences of radar signals, and calibrating the signal processing to produce a much cleaner input for the model.

The designed system does not only possess weaknesses but also many strengths. For example, the model, though relatively unstable, can and did in validation recognize many edge cases, such as walking, jumping, and slow sitting. Thus, it can be inferred that the foundation for a strong detection system is already laid, and the only shortcomings are the size of the dataset and sufficient background noise removal.

8. Conclusion

Overall, the system worked as required; meeting all but one requirement and demonstrating feasibility of the concept. While the actual performance metrics of the system could be improved, such as battery life, fall detection accuracy, and form factor, this would require extensive testing data, a different radar / microcontroller, and time.

For demonstrating the concept of using millimeter-wave radar to detect falls, the system worked as intended; showing both the pros and cons of this setup while also leaving the problem open for the development of a commercial ready product given the found results.